



SKILLNET: Create, Evaluate, and Connect AI Skills

Yuan Liang^{1*}, Ruobin Zhong^{1*}, Haoming Xu^{1*}, Chen Jiang^{3*}, Yi Zhong¹, Runnan Fang⁴, Jia-Chen Gu¹¹, Shumin Deng¹⁵, Yunzhi Yao¹, Mengru Wang¹, Shuofei Qiao¹, Yida Xue¹, Xin Xu¹², Tongtong Wu¹⁴, Kun Wang¹⁶, Yang Liu¹⁶, Zhen Bi¹⁷, Jungang Lou¹⁷, Yuchen Eleanor Jiang⁷, Hangcheng Zhu⁴, Gang Yu⁴, Haiwen Hong⁴, Longtao Huang⁴, Hui Xue⁴, Chenxi Wang¹, Yijun Wang⁶, Zifei Shan⁶, Xi Chen⁶, Zhaopeng Tu⁶, Feiyu Xiong¹⁰, Xin Xie⁵, Peng Zhang⁵, Zhengke Gui⁵, Lei Liang⁵, Jun Zhou⁵, Chiyu Wu⁸, Jin Shang⁸, Yu Gong⁸, Junyu Lin⁹, Changliang Xu¹⁹, Hongjie Deng¹, Wen Zhang¹, Keyan Ding¹, Qiang Zhang¹, Fei Huang¹⁸, Ningyu Zhang^{1†}, Jeff Z. Pan¹³, Guilin Qi³, Haofen Wang², Huajun Chen¹

¹Zhejiang University, ²Tongji University, ³Southeast University

⁴Alibaba Group, ⁵Ant Group, ⁶Tencent, ⁷OPPO, ⁸HomologyAI

⁹Fudan University, ¹⁰MemTensor (Shanghai) Technology, ¹¹UCLA, ¹²UCSD,

¹³The University of Edinburgh, ¹⁴Monash University, ¹⁵NUS, ¹⁶NTU, ¹⁷Huzhou University,

¹⁸Honor Device Co., Ltd, ¹⁹Hangzhou Institute for Advanced Study, UCAS

Abstract

Current AI agents can flexibly invoke tools and execute complex tasks, yet their long-term advancement is hindered by the lack of systematic accumulation and transfer of skills. Without a unified mechanism for skill consolidation, agents frequently “reinvent the wheel”, rediscovering solutions in isolated contexts without leveraging prior strategies. To address this challenge, we introduce **SkillNet**, an open infrastructure for creating, evaluating, and organizing AI skills at scale. SkillNet structures skills within a unified ontology that supports creating skills from heterogeneous sources, establishing rich relational connections, and performing multi-dimensional evaluation across Safety, Completeness, Executability, Maintainability, and Cost-awareness. Our infrastructure integrates a repository of over 600,000 skills, an interactive platform, and a versatile Python toolkit. Experiments on ALFWorld, WebShop, and ScienceWorld show 40% higher average rewards and 30% fewer execution steps across multiple backbone models. Furthermore, **SkillNet-Gym** benchmarks skill retrieval, utilization, and composition, while **SkillNet-Fabric** enables task-specific skill routing through lightweight Wikis. By formalizing skills as evolving, composable assets, SkillNet provides a robust foundation for agents to move from transient experience to durable mastery.

Correspondence: liang_yuan@zju.edu.cn, zhangningyu@zju.edu.cn

Website: <http://skillnet.openkg.cn>, <https://github.com/zjunlp/SkillNet>

*Equal Contribution.

†Corresponding authors.



Figure 1 Overview of **SkillNet**. SkillNet organizes large-scale agent skills into a structured skill network, modeling rich relations (e.g., similarity, composition, and dependency), supporting multi-dimensional evaluation, and providing unified interfaces for skill discovery, creation, and analysis.

1 Introduction

As Richard S. Sutton noted, “we are in the era of experience” [1]. Intelligence is increasingly grounded not in *ab initio* knowledge acquisition, but in the efficient retrieval and adaptive reuse of heuristics distilled from prior experience. This transition marks the advent of the agent era, where AI moves beyond static question answering to orchestrate long-horizon, executable tasks [2, 3, 4, 5, 6, 7, 8]. Despite recent progress in agentic systems, a critical yet underexplored question remains: **How can agents systematically consolidate episodic experience into durable, transferable mastery?**

Current AI methods largely rely on manual engineering or transient in-context learning [9, 10, 11]. While humans excel at consolidating episodic experience into reusable schemas [12], for instance when a programmer internalizes the logic of an algorithm rather than merely memorizing its syntax, contemporary AI systems struggle to bridge the divide between transient context and long-term capability. Without a unified mechanism for skill consolidation and sharing, agents repeatedly “reinvent the wheel” in isolated contexts, and even well-established strategies rarely inform future tasks. This gap highlights a broader, longstanding tension in AI between explicit structure and scalable representation, a limitation that becomes especially evident when examined through the historical evolution of knowledge engineering [13]:

- **The Symbolic Era:** Systems relied on rigid symbolic logic, offering interpretability but suffering from brittleness and limited scalability [14, 15, 16].
- **The Deep Learning Era:** Knowledge became parametric, taking the form of high-dimensional weight matrices. While powerful, such representations are opaque and difficult to modularize or reuse [17, 18].
- **The Agentic Era (Current Frontier):** We are witnessing a convergence toward agent skills, which serve as simple, transferable units that provide agents with new capabilities and expertise while separating intelligence from monolithic parameter spaces [19, 20, 21, 22].

This historical trajectory suggests a natural progression toward skills as modular, externalized units of knowledge that reconcile structural interpretability with scalable representation. Consequently, the central challenge in the agent era is no longer merely learning from experience, but transforming fragmented experience into durable, composable skill units that support generalizable intelligence. However, current AI approaches fall short in two fundamental respects. First, there is no unified mechanism to acquire and consolidate skills from experience. Valuable expertise exists at scale across open-source repositories, academic publications, and agent execution traces [23, 24], yet it remains largely unstructured and disconnected. Unlike human learners, who continuously internalize external information into organized knowledge schemas, AI agents cannot automatically distill these resources into reusable, executable capabilities. As a result, skill acquisition remains a manual and episodic process rather than an autonomous, cumulative one [3].

Second, there is no principled framework to validate and maintain skill quality at scale. Without intrinsic and systematic evaluation, skill repositories are prone to “pollution,” where executability, safety, and robustness are assessed only indirectly through downstream task success [25, 26, 27]. Such stochastic and opaque validation introduces technical debt, undermining long-term capability growth and preventing agents from leveraging accumulated experience in a reliable and scalable manner. Together, these gaps highlight the need for a structured framework that not only captures and organizes skills, but also rigorously evaluates and interconnects them to enable dependable, large-scale knowledge reuse.

To address these issues, we introduce **SkillNet** (Figure 1), an open infrastructure for creating, evaluating, and organizing AI skills at scale. We conceptualize a skill as a unified knowledge representation that bridges unstructured language understanding with structured, machine-executable logic. SkillNet constructs a comprehensive Skill Ontology with three interconnected layers: a taxonomic layer for functional categorization, a relational layer encoding dependencies and composition, and a skill-package layer for modular deployment. To ensure the reliability of these skills, we propose a multi-dimensional evaluation framework that assesses skills based on **Safety, Completeness, Executability, Maintainability, and Cost-awareness**.

Our approach transforms fragmented experience from diverse sources, including execution trajectories, open-source repositories, and documentation, into a structured network of over 500,000 curated skills. Experimental results across three text-based simulated environments (ALFWorld, WebShop, and ScienceWorld) demonstrate that agents augmented with SkillNet achieve substantial gains. For instance, our approach improves the average reward by 40% while reducing interaction steps by 30% across various backbone models (e.g., DeepSeek V3, Gemini 2.5 Pro, and o4 Mini), validating that systematic skill accumulation effectively enhances agent competence cumulatively rather than episodically.

Our contributions are summarized as follows:

1. We introduce SkillNet, a unified framework that transforms fragmented agent experience into a structured network of modular, composable skills with rich relational modeling, serving as a scalable foundation for actionable knowledge engineering.
2. We establish a rigorous skill evaluation protocol that quantitatively measures safety, completeness, executability, maintainability, and cost-awareness, ensuring the reliability of large-scale skill repositories.
3. We release an open-source ecosystem, including a repository of over 500,000 curated skills, a Python toolkit, and comprehensive benchmarks, which empirically demonstrate significant performance improvements in agent planning and execution tasks.
4. We introduce SkillNet-Gym, a dynamic benchmark automatically constructed from real-world skill ecosystems to evaluate skill construction, retrieval, and composition. Experiments across diverse frontier models and agent harnesses reveal that current agents remain limited in open-world skill utilization, particularly in skill retrieval and multi-skill orchestration.
5. We develop SkillNet-Fabric, a task time routing layer that places a task specific Wiki between the global skill ecosystem and final set formation. Evaluations on routing and downstream benchmarks demonstrate improved routing quality and task performance across the tested settings.

2 Agent Skills

In the context of agentic systems, **skills** represent a lightweight, modular, and reusable abstraction for extending the capabilities of AI agents [28]. Conceptually, a skill encapsulates procedural knowledge, task-specific instructions, and supporting resources, enabling agents to perform complex tasks more accurately, efficiently, and consistently.

Functionally, skills are organized as structured folders containing a central `SKILL.md` file, which defines the skill’s metadata and detailed instructions. The metadata typically includes the skill’s name, a brief description of its purpose, and usage conditions, while the instructions provide step-by-step guidance for execution. Skills may optionally include scripts, templates, documentation, and other resources required to perform tasks, forming a self-contained capability package.

The core purpose of skills is to provide agents with access to reusable procedural knowledge and context-specific information on demand. For example, an agent can leverage a skill to automate a data analysis pipeline, perform domain-specific reasoning, or generate structured outputs such as reports or presentations.

By encapsulating these capabilities, skills enable consistent and repeatable workflows, reduce reliance on hard-coded rules or ad hoc prompts, and allow knowledge to be shared across agents, teams, or applications. Skills operate through a progressive three-step process:

1. **Discovery:** Agents initially load only minimal metadata (e.g., name and description) to identify potentially relevant skills for a given task.
2. **Activation:** When a task matches a skill’s description, the agent reads the full instructions from `SKILL.md` and prepares any associated resources.
3. **Execution:** The agent follows the instructions and optionally executes bundled code or utilizes referenced assets to complete the task.

One of the key benefits of skills is that they are self-documenting. The embedded instructions and structured metadata make it straightforward to understand, audit, and iteratively improve each capability. Fundamentally, a skill serves as **a unified knowledge representation that integrates entities, relationships, workflows, and executable code, encompassing both textual semantics and symbolic outcomes**. This hybrid representation enables skills to bridge unstructured language understanding with structured, machine-executable logic. Skills are highly extensible, ranging from lightweight textual guidance to complex bundles that include executable programs and supporting assets. Their file-based, version-controlled nature ensures portability, allowing the same skill to be shared, reused, and deployed across different agents or systems with minimal adaptation.

Note that by encapsulating domain-specific knowledge and operational workflows in a structured and reusable form, skills provide an effective mechanism for institutionalizing business logic. This structured accumulation of expertise not only supports the systematic propagation of domain knowledge, but also enhances agent performance through reusable, reliable, and composable capabilities. However, in practice, these skills are often fragmented, scattered across scripts, prompts, or isolated workflows, and their quality, reliability, and reusability are rarely evaluated in a systematic way. Without a unified framework to organize, verify, and evolve skills, agents cannot fully realize the potential of this modular knowledge, limiting their ability to evolve continuously and adapt to new tasks. This motivates the development of SkillNet.

3 SkillNet

3.1 Overview

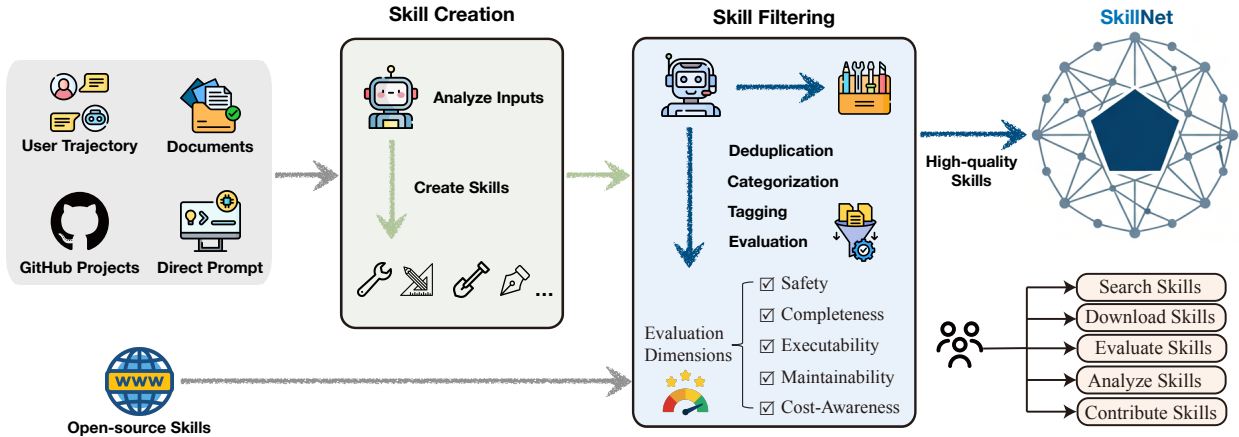


Figure 2 End-to-end Pipeline of SkillNet. SkillNet transforms heterogeneous user inputs and open internet resources into executable skills through automated skill creation and multi-dimensional evaluation, and organizes high-quality skills into a structured network to support search, download, analysis, and contribution.

Figure 2 illustrates the overall architecture of SkillNet, which systematically creates, evaluates, and organizes high-quality skills for agent systems. SkillNet is designed to transform fragmented agent experiences and human knowledge into reusable and verifiable skill entities, enabling scalable and reliable capability growth. SkillNet consists of three core modules:

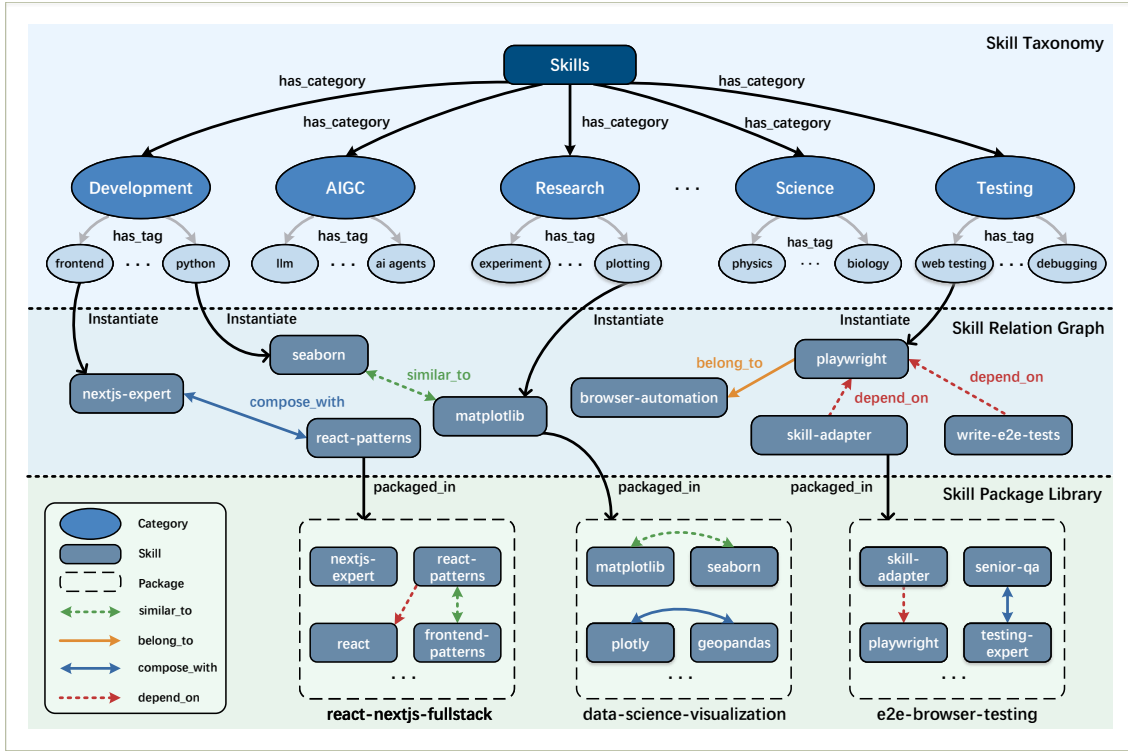


Figure 3 The Skill Ontology for SkillNet. It consists of three levels: the **Skill Taxonomy** (top) defines functional categories; the **Skill Relation Graph** (middle) models inter-skill dependencies and semantic associations; and the **Skill Package Library** (bottom) organizes skills into modular, task-oriented bundles.

Skill Creation. This module analyzes diverse inputs, including user trajectories, office documents, GitHub projects, direct prompts, and open internet resources. Based on these inputs, SkillNet generates new skills by extracting executable patterns and structuring them as reusable capabilities.

Skill Evaluation. Generated skills are filtered and evaluated along multiple principal dimensions, including safety, completeness, executability, maintainability, and cost-awareness. This evaluation ensures that only high-quality skills are retained, mitigating redundancy, brittleness, and potential risks.

Skill Analysis. Beyond isolated skills, SkillNet automatically analyzes structural and functional relationships among skills, constructing large-scale skill graphs that capture similarity, hierarchy, composition, and dependency patterns. This structured representation enables global reasoning over skill repositories, supporting efficient retrieval, composition, and workflow synthesis.

Open Resources. SkillNet organizes curated, high-quality skills within a structured repository, offering versatile toolkits that enable users and agents to efficiently search, download, create, evaluate, analyze, and contribute skills. By providing these standardized interfaces for skill interaction, SkillNet facilitates seamless skill reuse and collaborative evolution across tasks, domains, and agent populations.

3.2 Skill Ontology

The Skill Ontology organizes individual skills into a structured, composable network, as illustrated in Figure 3. The architecture is organized into three progressive layers:

Skill Taxonomy. This layer organizes skills into a multi-level hierarchy using category and tag relations. It categorizes broad domains (e.g., Development, AIGC, Science ...) into fine-grained tags (e.g., frontend, llm, physics), providing a high-level semantic skeleton.

Skill Relation Graph. This layer instantiates abstract tags into specific *Skill Entities* (e.g., Matplotlib, Playwright). It defines the core interaction logic through multi-relational edges: `similar_to`, `compose_with`, `belong_to`, and `depend_on`, forming the backbone for reasoning and planning.

Skill Package Library. The bottom layer represents the physical organization of skills. Individual skills are encapsulated into *Skill Packages* (e.g., `data-science-visualization`) via the `packaged_in` relation, facilitating modular release and deployment.

Note that the skill ontology is dynamic, modeling relations between skills. Tags can be continually added from the taxonomy, and LLMs infer relations from them, enabling the instantiation of a skill relation graph.

3.3 Skill Creation

SkillNet conceptualizes skills as intermediate capability units that bridge abstract, declarative knowledge and concrete, executable programs. Specifically, we develop an automated creation pipeline that systematically transforms heterogeneous information sources into standardized, reusable agent skills.

3.3.1 From Human Experience to Skill Abstraction

To construct a comprehensive and versatile skill repository, SkillNet abstracts skills from diverse data sources, transforming heterogeneous information into reusable structured agent skills.

We design a multi-source automated skill creation pipeline that enables SkillNet to induce agentic skills from heterogeneous human knowledge and experience. Specifically, SkillNet supports four major categories of data sources: (1) execution trajectories and conversational interaction logs, (2) open-source GitHub repositories, (3) semi-structured documents including PDF, PowerPoint, and Word files, and (4) direct natural language prompts provided by users. This process is fully implemented through LLMs, and users can also customize the underlying models. In addition, SkillNet continuously expands its skill pool through open internet resources, in-house development, and community contributions, ensuring sustained scalability and coverage expansion.

3.3.2 Data-Driven Skill Filtering and Consolidation

Automatic construction of skills does not imply indiscriminate accumulation. Instead, SkillNet introduces a data-driven filtering and consolidation pipeline to ensure the quality of the skill repository. Specifically, SkillNet applies a multi-stage curation process consisting of deduplication, filtering, categorization and tagging, evaluation, and final selective consolidation.

Deduplication is performed by jointly comparing skill directory structures and MD5 hashes of skill markdown files, effectively removing redundant skills. Filtering eliminates low-quality, incomplete, or semantically meaningless skills through rule-based validation and model-based checking. Categorization and tagging classify each skill into one of ten functional categories (Development, AIGC, Research, Science, Business, Testing, Productivity, Security, Lifestyle, Other) and assign fine-grained semantic tags to facilitate retrieval and composition. Then, a multi-dimensional evaluation mechanism is applied to determine whether a skill is admitted into the SkillNet repository. SkillNet automatically establishes inter-skill associations based on ontology-defined relations, ultimately resulting in a structured skill package library.

This structured pipeline enables SkillNet to function as a self-evolving skill ecosystem rather than a static skill collection, continuously improving both skill quality and coverage.

3.4 Skill Evaluation

Although skill creation is important, the utility of a skill repository ultimately depends on its reliability. To bridge the gap left by existing repositories that lack standardized assessment, we propose a multi-dimensional systematic evaluation framework.

We define five core dimensions to quantitatively characterize the quality and readiness of each skill:

- **Safety:** Assesses potential risks, including hazardous system operations (e.g., unauthorized file deletion) and robustness against prompt injection or adversarial manipulation.
- **Completeness:** Evaluates whether the skill encapsulates all critical procedural steps and explicitly defines necessary prerequisites, dependencies, and execution constraints.

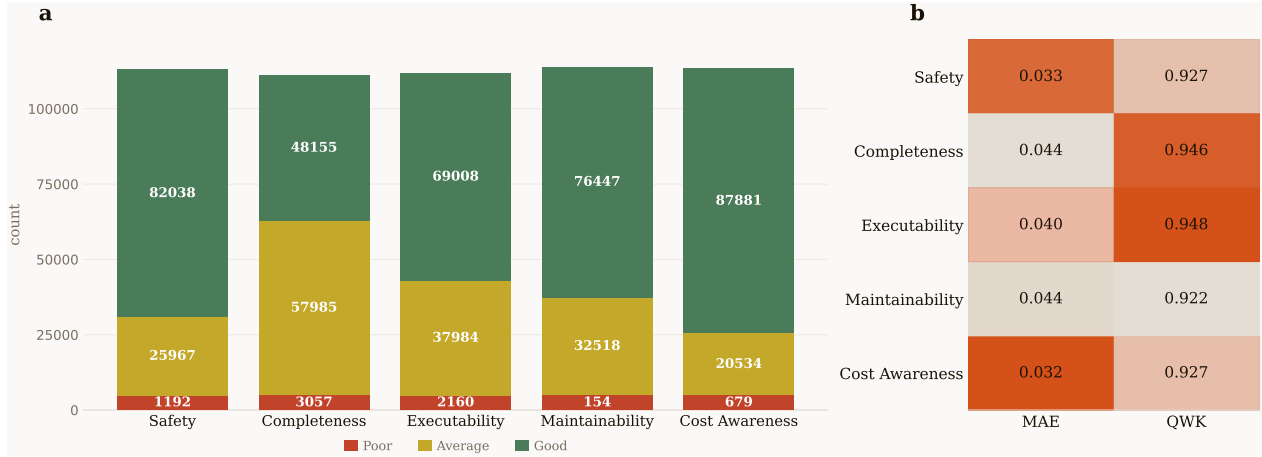


Figure 4 Multi-dimensional Skill Evaluation and Reliability Analysis. (a) Quality distribution of curated skills in SkillNet, assessed using a three-level grading scheme, where most skills are rated *Good* or *Average*. (b) Reliability validation of the automated evaluator on 200 randomly sampled skills. The heatmap shows mean absolute error (MAE) and quadratic weighted kappa (QWK) between human annotators and the model evaluator, with near-perfect QWK and low MAE across all dimensions, demonstrating the robustness and scalability of the evaluation framework.

- **Executability:** Verifies if the skill can be successfully implemented by agents in sandboxed environments, identifying hallucinated tool calls or ambiguous instructions.
- **Maintainability:** Measures the modularity and composability of skills, ensuring they can be locally updated without disrupting global dependencies or breaking backward compatibility.
- **Cost-awareness:** Quantifies execution overhead, including time latency, computational resource consumption, and API usage costs to support efficiency optimization.

To achieve high-throughput assessment, all dimensions are primarily evaluated via an automated LLM-based evaluator, guided by fine-grained rubrics. For **Safety**, the evaluator assesses potentially risky instructions and operations at the semantic level, providing a scalable repository-level signal for skill curation. This assessment is intended to support large-scale quality control rather than substitute for deployment-level security validation. For **Executability**, we complement LLM judgment with empirical validation: skills containing code or tool invocations are executed in controlled sandbox environments to verify runtime correctness. Each dimension is categorized into three levels: *Good*, *Average*, and *Poor*.

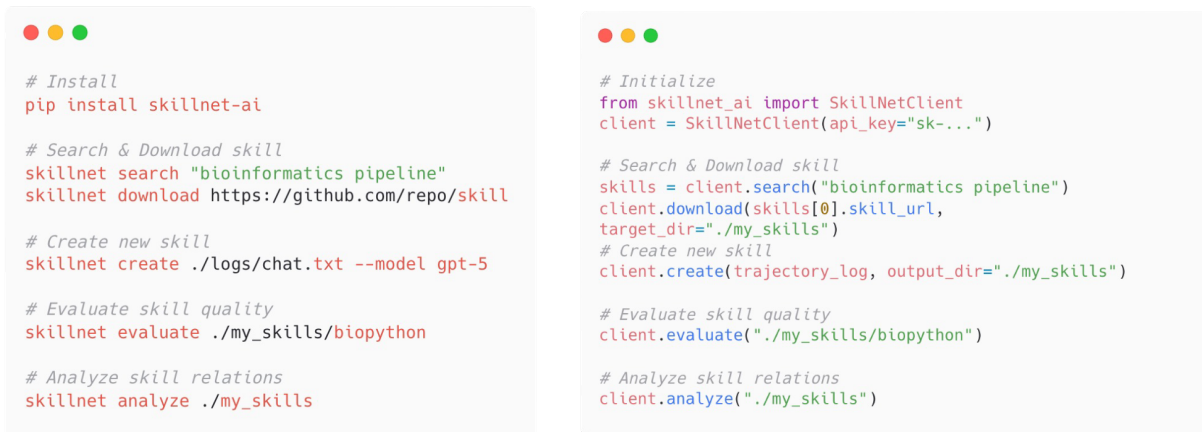
The quality distribution across the repository is illustrated in Figure 4(a). We observe that while **Safety** and **Maintainability** maintain a high proportion of “Good” ratings, **Executability** presents a greater challenge, with a larger fraction of skills rated as *Average*. This reflects our rigorous filtering criteria, ensuring that only high-fidelity skills are prioritized for complex task execution.

To validate the reliability of this automated pipeline, we randomly sampled 200 skills and recruited three PhD-level computer science annotators for independent blind reviews. As shown in Figure 4(b), the inter-rater agreement between human judgments and LLM-based scores exhibits exceptional consistency. Across all dimensions, the Mean Absolute Error (MAE) remains below 0.03, and the Quadratic Weighted Kappa (QWK) consistently reaches near-perfect levels (above 0.90). These results confirm that our automated evaluator provides a human-aligned, robust, and scalable foundation for managing the SkillNet ecosystem.

3.5 Skill Analysis

Beyond isolated skill creation and evaluation, large-scale skill repositories introduce a new challenge: how to systematically understand, organize, and exploit the relations among skills. To address this, SkillNet introduces a dedicated Skill Analysis module that automatically discovers and models structural relations between skills, forming a structured and interpretable skill relation graph. This enables global reasoning over large skill repositories and supports advanced downstream applications such as skill retrieval, composition, dependency resolution, and workflow synthesis.

Relation Modeling. SkillNet formulates skill analysis as a structured relations discovery problem. Given a large amount of heterogeneous skills, the system automatically identifies and annotates multiple types of semantic and functional relations between skills, including (task-specific relations are omitted here, and users



```
# Install
pip install skillnet-ai

# Search & Download skill
skillnet search "bioinformatics pipeline"
skillnet download https://github.com/repo/skill

# Create new skill
skillnet create ./logs/chat.txt --model gpt-5

# Evaluate skill quality
skillnet evaluate ./my_skills/biopython

# Analyze skill relations
skillnet analyze ./my_skills
```

```
# Initialize
from skillnet_ai import SkillNetClient
client = SkillNetClient(api_key="sk-...")

# Search & Download skill
skills = client.search("bioinformatics pipeline")
client.download(skills[0].skill_url,
target_dir="./my_skills")
# Create new skill
client.create(trajecory_log, output_dir="./my_skills")

# Evaluate skill quality
client.evaluate("./my_skills/biopython")

# Analyze skill relations
client.analyze("./my_skills")
```

Figure 5 Practical use examples of skillnet-ai. The package offers a unified functional experience through both a command-line interface (left) for interactive usage and a Python library (right) for seamless integration into AI development pipelines.

may extend the schema by defining custom relations as needed):

- **similar_to:** Two skills perform functionally equivalent or highly similar tasks and can often be used interchangeably, enabling redundancy detection, replacement, and robustness enhancement.
- **belong_to:** A skill serves as a sub-component or atomic step within a larger composite workflow, capturing hierarchical structures and supporting skill abstraction and modularization.
- **compose_with:** Two skills are frequently co-invoked in workflows, where one typically produces outputs consumed by the other, enabling automatic workflow composition and pipeline generation.
- **depend_on:** A skill cannot execute independently without prerequisite skills, such as environment setup or API initialization, enabling explicit dependency tracking and safe execution planning.

Automated Skill Relation Graph Construction. SkillNet can construct large-scale skill relation graphs through a hybrid pipeline that integrates semantic embedding and LLM-based relational inference. Candidate relations are first generated via similarity matching, dependency extraction, and execution trace alignment, and LLM reasoning, enabling scalable and accurate discovery of structured skill relations. These relations jointly form a directed, typed multi-relational graph, where nodes represent skills and edges encode fine-grained structural dependencies and functional associations.

Task-Oriented Skill Collection Release. To promote reproducibility and practical adoption, SkillNet releases over 20 task-specific skill collections, covering domains such as scientific research automation, software development, data processing pipelines, web automation, etc. In particular, several collections are designed for academic benchmarks, and we conduct quantitative evaluations to assess the effectiveness of this paradigm, as detailed in a later section. These released collections provide structured and reusable infrastructures for studying large-scale agent planning, workflow synthesis, and skill evolution, offering a foundation for future research on structured agent intelligence.

3.6 Open Resources

SkillNet provides a comprehensive open infrastructure for creating, evaluating, and organizing AI skills at scale. This includes a large-scale skill repository, a front-end website, an open access API, and a versatile Python toolkit (skillnet-ai), forming a unified ecosystem for skill management and utilization.

The front-end website allows users to seamlessly browse, search, and download skills directly. Users can explore skill categories, view detailed API documentation and Python toolkit usage guidelines, and learn

how to apply SkillNet to scientific research or coding scenarios. The platform also supports community contributions, enabling users to submit and share newly constructed skills, thus fostering a collaborative and

Model	Method	ALFWorld				WebShop		ScienceWorld			
		Seen		Unseen		R $\uparrow$	S $\downarrow$	Seen		Unseen	
		R $\uparrow$	S $\downarrow$	R $\uparrow$	S $\downarrow$			R $\uparrow$	S $\downarrow$	R $\uparrow$	S $\downarrow$
DeepSeek V3.2	React	66.43	19.51	69.40	19.27	<u>31.55</u>	24.06	69.86	17.59	64.67	19.26
	Expel	<u>67.86</u>	<u>18.86</u>	<u>76.12</u>	<u>17.41</u>	29.23	<u>24.00</u>	<u>74.91</u>	<u>15.98</u>	<u>74.09</u>	<u>17.53</u>
	+ SkillNet	80.60	14.54	83.57	14.81	46.18	17.84	84.87	11.89	81.31	12.48
Gemini 2.5 Pro	React	60.00	18.72	61.94	19.18	31.66	22.12	58.24	18.42	56.13	19.07
	Expel	<u>68.57</u>	<u>17.88</u>	<u>70.15</u>	<u>17.04</u>	<u>33.12</u>	<u>19.31</u>	<u>72.76</u>	<u>15.01</u>	<u>67.37</u>	<u>14.91</u>
	+ SkillNet	91.43	12.80	91.04	11.96	53.02	14.91	88.84	11.49	86.26	11.30
o4 Mini	React	45.71	23.31	49.25	23.33	24.19	22.02	64.89	15.12	59.93	14.99
	Expel	<u>56.43</u>	<u>21.35</u>	<u>58.96</u>	<u>21.85</u>	<u>26.71</u>	<u>21.91</u>	<u>67.95</u>	<u>13.65</u>	<u>65.68</u>	<u>13.95</u>
	+ SkillNet	68.57	18.94	73.28	17.08	36.21	18.79	73.24	13.30	71.06	12.35
Claude Sonnet 4.6	React	88.57	12.00	89.55	12.32	41.09	21.58	79.90	13.72	78.16	14.27
	Expel	<u>92.86</u>	<u>11.82</u>	<u>91.04</u>	<u>12.29</u>	<u>48.63</u>	<u>20.07</u>	75.99	<u>12.99</u>	76.24	14.88
	+ SkillNet	95.71	10.68	95.52	11.03	50.92	19.14	83.25	10.77	82.94	11.91

Table 1 Experimental results on ALFWorld, WebShop, and ScienceWorld. R denotes average reward, S denotes average steps ($\uparrow$ indicates the larger values are better, and $\downarrow$ denotes the smaller values are better). The top-performing results are highlighted in **bold**, and the second-best are underlined.

continuously evolving skill ecosystem.

For API access, SkillNet supports both keyword-based and vector-based searches. Users can search for skills by keywords, categories, or semantic similarity via <http://api-skillnet.openkg.cn/v1/search>.

To bridge the gap between static repositories and dynamic execution, we developed `skillnet-ai`, a unified Python library and CLI tool that enables flexible integration with users. Through this toolkit, users can search for skills using keywords or semantic similarity, download skills directly from GitHub into local workspaces, and create structured skills from heterogeneous sources, including execution trajectories, GitHub repositories, office documents, and natural language prompts. The package also supports evaluating skills across multiple dimensions (safety, completeness, executability, maintainability, and cost-awareness) to ensure practical reliability, and analyzing relations between skills to uncover dependencies, hierarchical compositions, collaborations, and functional similarities. The practical use examples are illustrated in Figure 5.

SkillNet has initially aggregated over 600k candidate skills from open internet resources, automated creation pipelines, and community contributions. After multi-stage filtering and evaluating, more than 500k high-quality skills are curated in the final repository (and which is constantly expanding). This scale ensures broad coverage across scenarios while maintaining rigorous quality standards, enabling SkillNet to serve as a reliable skill infrastructure for both scientific investigation and real-world deployment. We actively encourage community contributions, and all community-submitted skills are subject to the same automated quality checks. In addition, we conduct periodic manual audits through random sampling to ensure the ongoing reliability and integrity of the shared skill repository.

4 Quantitative Evaluation

4.1 Settings

To quantitatively assess the effectiveness of SkillNet, we conduct experiments across three text-based simulated environments. ALFWorld [29] provides an embodied household environment, requiring agents to navigate and manipulate objects to complete daily tasks; WebShop [30] simulates realistic online shopping scenarios, where agents perform product search, comparison, and purchasing under specified constraints; and ScienceWorld [31] presents a virtual scientific laboratory in which agents must conduct experiments and operate scientific instruments. All three environments are formulated as partially observable Markov

decision processes (POMDPs), requiring agents to make sequential decisions under incomplete and noisy observations.

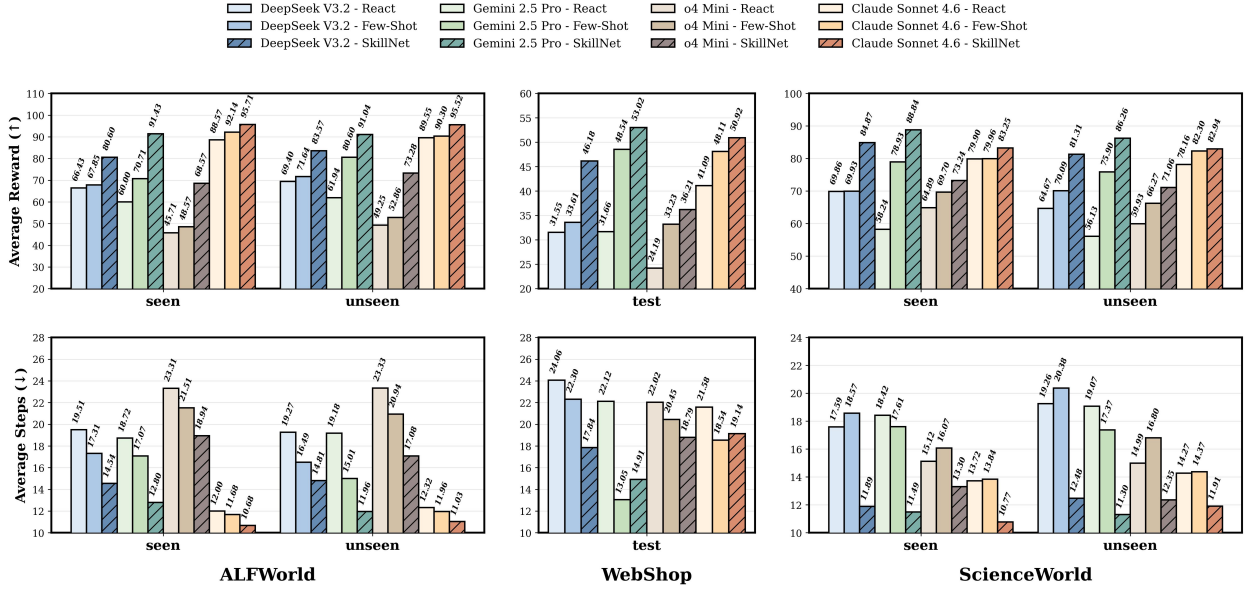


Figure 6 Performance comparison across diverse methods and models. The results illustrate that SkillNet consistently outperforms React and Few-shot baselines, achieving significantly higher average rewards (top) and reduced average steps (bottom) across ALFWorld, WebShop, and ScienceWorld.

We adopt ReAct [32], Expel [20] and a standard Few-Shot approach as the baseline methods. ReAct interleaves reasoning, action, and observation for step-by-step task solving. Expel gathers experiences from past tasks to extract natural language insights and retrieves past successful trajectories as in-context examples during inference. The Few-Shot baseline randomly samples a complete expert trajectory to serve as a static in-context exemplar. In contrast, our proposed variant is augmented with SkillNet by leveraging expert trajectories from ETO [33] to synthesize benchmark-specific skill collections (these collections are integrated into SkillNet, and users can extend it to other datasets by creating corresponding skills)¹. Note that the above experiences and the seen and unseen splits of the test set have no overlap, thereby preventing data leakage. During evaluation, agents are equipped with these collections and can dynamically select, activate, and execute the most relevant skills based on the current state. We employ four representative LLMs: DeepSeek V3.2 [34], Gemini 2.5 Pro [35], o4 Mini [36], and Claude Sonnet 4.6 [37] as backbone models.

4.2 Results

Table 1 and Figure 6 present the quantitative results across diverse environments and backbone models. In general, integrating SkillNet consistently yields substantial gains in both task effectiveness and execution efficiency, validating the effectiveness of our framework. Compared with ReAct, SkillNet improves the average reward by 40%, while reducing the number of interaction steps by 30% on average, indicating that agents equipped with SkillNet are able to solve tasks more reliably while executing shorter and more coherent action trajectories. This reflects SkillNet’s ability to transform fragmented experience into reusable procedural abstractions, enabling agents to perform structured planning, reduce redundant exploration, and execute complex behaviors in a more principled and systematic manner.

Importantly, the performance improvements remain robust across backbone models of varying capacities, ranging from compact models (+15.7 R for o4 Mini) to large-scale LLMs (+28.5 R for Gemini 2.5 Pro). This demonstrates that SkillNet provides complementary capabilities beyond parametric knowledge, effectively augmenting reasoning with persistent, executable skills. Moreover, the consistent gains observed under both seen and unseen settings highlight SkillNet’s strong generalization ability, suggesting that skill abstraction and reuse facilitate knowledge transfer across tasks and environments.

¹<https://github.com/zjunlp/SkillNet/tree/main/experiments/src/skills>.

Taken together, these results substantiate the central design principle of SkillNet: by formalizing skills as independent, systematically accumulated, knowledge-grounded capability units, agent competence can be enhanced cumulatively rather than episodically, ultimately fostering domain-specialized and continually self-improving agents.

5 SkillNet-Gym

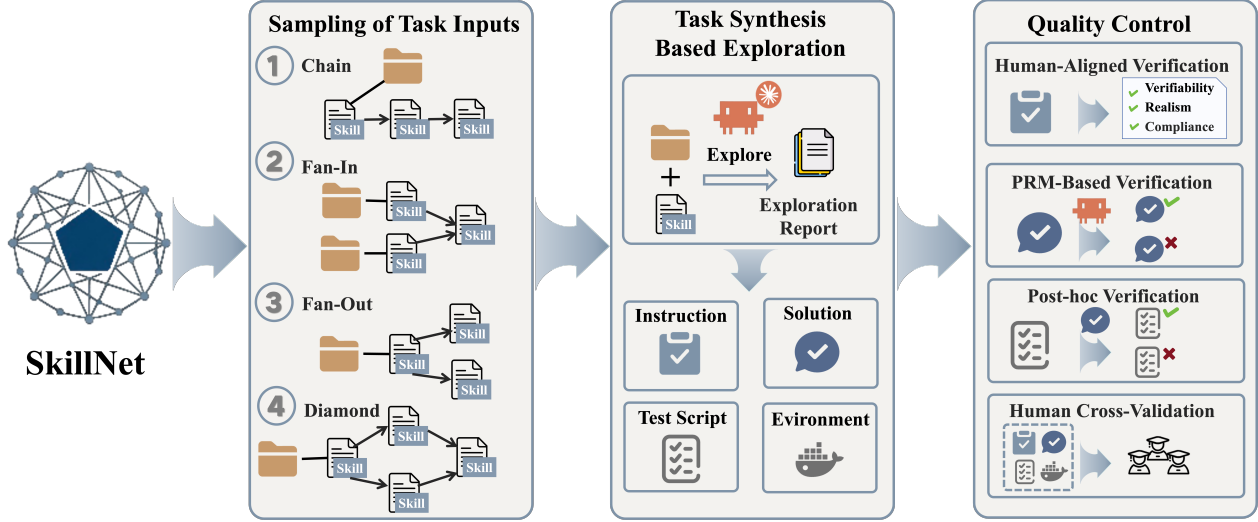


Figure 7 The overview of SkillNet-Gym construction pipeline.

5.1 Design Principles

To reflect the demands of realistic and continuously evolving community, we design SkillNet-Gym around four key properties: (1) Real-World Source: SkillNet-Gym is grounded in artifacts collected from real community resources, rather than manually curated skills or synthetic files. This design choice ensures that the benchmark better reflects the structure, variability, and messiness of real-world usage scenarios. (2) Domain diversity: SkillNet-Gym spans 13 core domains and 81 subdomains, covering a broad range of practical settings, including data analysis, science, technology and so on. (3) Dynamic extensibility: To accommodate the continuously evolving community ecosystem, SkillNet is represented as a heterogeneous graph that can be incrementally expanded by adding new artifact nodes and their associated edges. This formulation enables the benchmark to evolve over time. (4) Automatic construction: Our pipeline automatically constructs the benchmark from SkillNet at scale, and also supports automatic control over task difficulty by varying the number of nodes visited during random walks and the topological structures induced by the traversal. The comparison of SkillNet-Gym with other benchmarks is detailed in Table 2. Next, we introduce the automated pipeline of SkillNet-Gym, covering SkillNet construction, benchmark data synthesis, and quality control. The pipeline is shown in Figure 7.

5.2 Skill Graph Construction

We construct skill graph (customized SkillNet) from real community resources by collecting three aligned artifact types: skills, supporting documents, and processable files. Specifically, we retrieve candidate skills from repositories such as ClawHub and SkillHub using LLM-generated domain-diverse queries, then perform filtering and deduplication. For each skill, we gather web documentation based on its functional semantics, or synthesize surrogate documentation when necessary, and retain functionally compatible files with source metadata for downstream normalization and quality control. On top of these artifacts, we build SkillNet as a skill-centric DAG: an LLM extracts each skill’s pre and post scenarios, we identify candidate edges by matching embedded post-scenarios to pre-scenarios, and then use an LLM to verify whether each match constitutes a valid workflow handoff. The resulting graph captures directed compositional dependencies

Benchmark	Benchmark Construction			Benchmark Scope			Evaluation Focus	
	Real-World Source	Automatic Construction	Dynamic Extensibility	Domain Coverage	Skill Pool	Task Numbers	Skill Construction	Skill Composition
SkillsBench [38]	✓	✗	✗	✓	✗	87	✗	✗
SWE-Skills-Bench [39]	✓	✗	✗	✗	✗	565	✗	✗
SkillLearnBench [40]	✓	✗	✗	✓	✗	20	✓	✗
WildSkillBench [41]	✓	✗	✗	✓	✓	87	✓	✓
SkillRouter [42]	✓	✓	✓	✓	✓	75	✗	✓
SkillRet [43]	✓	✓	✓	✓	✓	4997	✗	✓
AgentSkillOS [44]	✓	✗	✗	✓	✓	30	✗	✓
SkillGenBench [45]	✓	✓	✓	✓	✗	187	✓	✗
OpenSkillEval [46]	✓	✓	✓	✓	✗	600	✗	✗
SkillNet-Gym (ours)	✓	✓	✓	✓	✓	159	✓	✓

Table 2 Comparison of existing agentic skills benchmarks across three dimensions: **Benchmark Construction**, **Benchmark Scope** and **Evaluation Focus**. “✓” indicates fully coverage, “✓” indicates partially coverage, and “✗” indicates not coverage. Detailed explanations for each trait are provided in Appendix A.

among skills and, compared with scenario-centric formulations, more directly models skill topology for multi-skill task synthesis.

5.3 Automatic Task Synthesis from SkillNet

Diversity Sampling of Task Inputs. Given the constructed SkillNet, we synthesize task candidates by sampling structured subgraphs from the directed skill graph. We consider four common composition patterns: chains, fan-out, fan-in, and diamonds, with two to five skills depending on the structure, as illustrated in Figure ?? . Each candidate preserves its participating skills, directed skill connections, and scenario-level evidence for every edge. We score each edge using alignment confidence, retrieval similarity, and the number of supporting scenario alignments, and score each task by combining mean edge quality with a structure bonus and a hub-node penalty. The resulting candidates are grouped by structure category and assigned difficulty labels from easy to expert according to their compositional complexity. We then select tasks either by quality ranking. This produces a pool of compositional task candidates grounded in verified SkillNet transitions.

Agent-Driven Exploration Report Generation. Given the sampled skill Directed Acyclic Graph (DAG) and its associated input artifacts, we prompt an agent (Claude Code) to execute the skills by following the workflow specified by the DAG and operating directly on the input files. After the exploration phase, the agent produces an exploration report that summarizes actionable findings, such as feasible cross-skill workflows, potential intermediate artifacts, and common failure patterns. Compared with naively concatenating the full contents of all skill files, the exploration report is substantially more concise and better aligned with downstream task requirements.

Task Synthesis. A complete task should include at least four core components: a task instruction, a golden solution, an evaluation script, and the execution environment. We synthesize these components sequentially. 1) *Task Instruction:* Given the exploration report and snippet-level information about the input artifacts, we prompt an LLM to generate a task instruction that follows the workflow induced by the skill DAG. To encourage diversity, each synthesis run uses a randomly sampled writing style. 2) *Golden Solution:* After generating the instruction, we first derive high-level guidance from the exploration report to scaffold the downstream problem-solving process. We then prompt Claude Code to solve the task conditioned on this meta-information. From the resulting execution trajectory, we extract and compress the essential steps into a canonical reference solution, `solve.sh`. 3) *Test Script:* Based on the task instruction and the final output artifacts, we synthesize a test script `test.sh` that verifies not only the structural validity of the outputs but also their semantic correctness at the value level. 4) *Execution Environment:* Finally, we extract the minimal execution environment to get a Dockerfile from `solve.sh`.

5.4 Quality Control

We apply rigorous multi-stage quality control to the benchmark. First, during SkillNet construction, we filter low-quality nodes and edges to improve the reliability of the underlying skill graph. Second, for synthesized tasks, task-level quality control covers four core components of each instance: the instruction, the reference

Models	Skill Construction						Skill Composition			
	No Skill		Official Skill		Skill-Creator		Skills Retrieve		Skills Orchestration	
	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard
Claude Code										
Minimax-M2.7	61.04	17.89	65.58	26.22	66.88	20.73	8.12	13.10	18.55	11.62
Kimi-K2.6	70.13	26.83	74.03	34.15	73.38	<u>31.10</u>	14.29	19.51	4.62	12.66
Deepseek-V4	65.28	23.38	69.44	29.87	68.83	29.88	<u>24.68</u>	<u>30.49</u>	22.22	<u>24.36</u>
GLM-5.1	70.13	26.83	71.43	32.93	79.22	31.10	16.02	25.20	8.72	20.68
Claude 4.5 Haiku	59.09	16.46	55.84	26.22	53.90	23.17	15.21	21.65	8.72	9.33
Claude 4.6 Sonnet	62.34	20.73	74.03	<u>36.59</u>	72.73	30.49	25.98	39.17	<u>14.59</u>	26.70
Codex										
GPT-5.4 mini	<u>72.73</u>	<u>26.83</u>	<u>74.03</u>	32.93	<u>75.32</u>	30.49	5.63	14.63	6.67	15.19
GPT-5.4	74.68	28.66	75.32	37.20	74.03	26.83	11.26	18.29	11.35	16.46

Table 3 Comparison across different harnesses and models under different skill usage settings. **No Skill** uses no external skills; **Official Skill** directly provides the task-associated official skills; **Skill-Creator** evaluates execution with skills constructed from upstream resources; **Skills Retrieve** requires agents to retrieve relevant skills from a large wild skill pool before execution; **Skills Orchestration** additionally requires dependency-aware organization of the selected skills into an execution workflow. Best and second-best results within each column are shown in **bold** and underlined, respectively.

solution, the test script, and the task difficulty. Through this rigorous quality control pipeline, we ultimately retain 159 high-quality tasks. Detailed quality control procedures are provided in Appendix C.1.

5.5 Difficulty Stratification for Analysis

For analysis, we further partition the tasks into two coarse difficulty strata, **Easy** (77 tasks) and **Hard** (82 tasks), using post hoc empirical performance as a calibration signal while maintaining balanced domain coverage across the two subsets. This stratification enables more fine-grained comparisons of model behavior across tasks with different realized levels of difficulty.

5.6 Evaluation Methodology and Metrics

Skill Construction. We study agent behavior from the perspective of skill lifecycle: (1) **No Skill**: In the initial stage, the agent solves tasks without access to any skills. (2) **Official Skill**: The agent can leverage externally provided skills, such as official skills released in community repositories like SkillHub and ClawHub. (3) **Skill-Creator**: Using meta-skill, e.g., Skill-Creator, the model distills human-authored procedural documents into reusable skills that it can invoke in future task execution. In the final setting, the agent executes tasks end-to-end under both settings: with skills and without skills. For evaluation, each execution trajectory receives a reward of 1 if the agent’s final output passes all unit tests, and 0 otherwise. Each task is attempted independently for k runs, and we report **Avg@ k** .

Skill Composition Beyond end-to-end evaluation, we also consider a finer-grained setting designed to assess how an agent retrieves and orchestrates skills from a realistic skill pool to solve a given task. In this setting, the agent is not required to fully execute each task. Instead, we focus on its ability to identify appropriate skills from the skill pool, which is constructed from the 2,000+ skills included in SkillNet and organize them into an effective execution workflow. We report two stage-specific completeness metrics: (1) **Skill Retrieval Completeness**, which measures whether the retrieved skill nodes fully cover the gold skill set; (2) **Skill Orchestration Completeness**, which measures whether the predicted dependency edges fully recover the gold workflow among retrieved skills. Both metrics use binary completeness because our goal is to evaluate whether an agent can recover a complete executable composition rather than partial overlap.

5.7 Experimental settings

Agent Harnesses. We evaluate the two most popular agents in current use: Claude Code [47] and Codex CLI [48].

Models. We evaluate eight frontier models, including four open-source models and four proprietary models: MiniMax-M2.7 [49], Kimi-K2.6 [50], Deepseek-V4 [51], GLM-5.1 [52], Claude Haiku 4.5, Claude Sonnet 4.6, GPT-5.3 Codex, and GPT-5.4. All models are evaluated with their default temperature settings, and each task is independently run three times. Following commonly adopted community practices, we pair the open-source models and the Claude series with Claude Code, while the GPT series is paired with Codex.

5.8 Main Results

Skills Help Hard Tasks More Reliably Than Easy Tasks. Compared with the easy tasks, skill augmentation generally yields larger and more consistent relative improvements on the hard tasks. A likely reason is that current frontier models already possess strong agentic capabilities, even without external skills, they can often solve many easy tasks reasonably well. Consequently, the headroom for improvement on easy tasks is limited, whereas challenging tasks benefit more substantially from explicit skill support. The corresponding results are reported in Table 3.

One-Shot Skill Construction May Introduces Information Loss. Although the model is provided with documents of higher information density, compressing them into a skill inevitably discards part of the original information, which can lead to degraded performance for tasks requiring specific capabilities. The corresponding evidence is shown in Table 3. At the same time, our results also suggest that officially curated skills are not always superior to self-constructed ones, highlighting the importance of model-side adaptation. In addition, we evaluate another skill-construction baseline, SkillSeeker [53], whose results are reported in Table 9 and exhibit similar trends. While our current study only considers a single-pass compression strategy, existing skill-construction methods are increasingly moving toward more sophisticated pipelines [54, 55, 56] that enable adaptation and self-evolution. We believe that, in the future, model constructed skills may eventually surpass currently human curated skills.

Frontier Agents Exhibit Weak Skill Composition Ability in the Wild. Although frontier agents can benefit from provided skills in controlled settings, their ability to operate over large, community-sourced skill repositories remains limited. Table 3 reveals a notable difficulty inversion between skill retrieval and skill orchestration. Hard tasks often obtain higher retrieval scores than easy tasks, suggesting that complex instructions expose richer procedural cues and make the required skills more identifiable. However, this advantage does not translate into successful orchestration. Once the agent must organize the retrieved skills into a dependency-preserving workflow, hard tasks become substantially more challenging. We also evaluate end-to-end execution in a realistic wild setting in Table 10. Compared with directly providing official skills, exposing agents to a large skill library leads to clear performance degradation, with the drop being more pronounced on hard tasks. These results show that incomplete retrieval and incorrect orchestration are not only intermediate failures, but directly limit downstream task success. Therefore, effective real-world skill use requires agents to go beyond consuming fixed skills and develop adaptive orchestration mechanisms that can expand exploration, verify dependencies, and update skill workflows as the skill ecosystem evolves.

5.9 Analysis

Specialized Composition Baselines Remain Stronger than Agents. We further evaluate representative baselines specifically designed for skill composition, including SkillRouter [42], SkillRet [43], and AgentSkillOS [44], and compare them against the average performance of the eight agents on Top-10. The results are shown in Table 4. Here, Avg. sel denotes the average number of skills retrieved and the average number of dependency edges selected by the agent. Among the retrieval baselines, SkillRet achieves substantially higher recall than SkillRouter, despite the latter using a reranker, likely due to SkillRet’s access to larger-scale training data. For AgentSkillOS, we do not adopt its original high-cost tree-construction pipeline; instead, we prompt GLM-5.1 to perform selection and orchestration based only on the preliminary candidates returned by SkillRet. AgentSkillOS substantially outperforms direct skill orchestration from the skill repository. These results

Methods	Complete	Recall	Precision	Avg sel.
Skills Retrieval				
Agent	18.83	44.93	37.92	3.86
AgentSkillOS	42.14	61.77	55.00	-
SkillRouter				
w/o Reranker	36.48	60.28	19.06	-
w/ Reranker	44.03	67.40	21.01	-
SkillRet	59.12	78.40	24.09	-
Skills Orchestration				
Agent	18.36	25.17	18.63	3.27
AgentSkillOS	29.49	35.51	26.86	-

Table 4 Performance of Skill Composition Baselines.

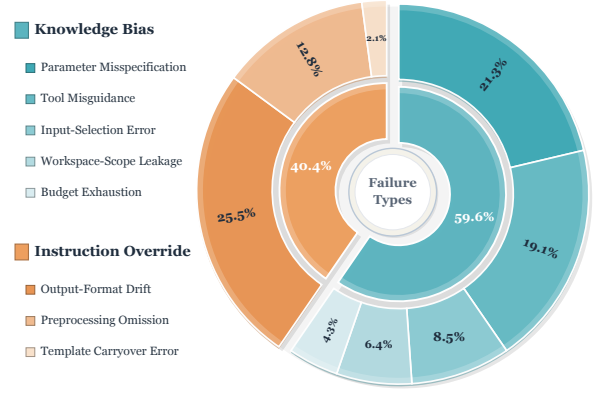
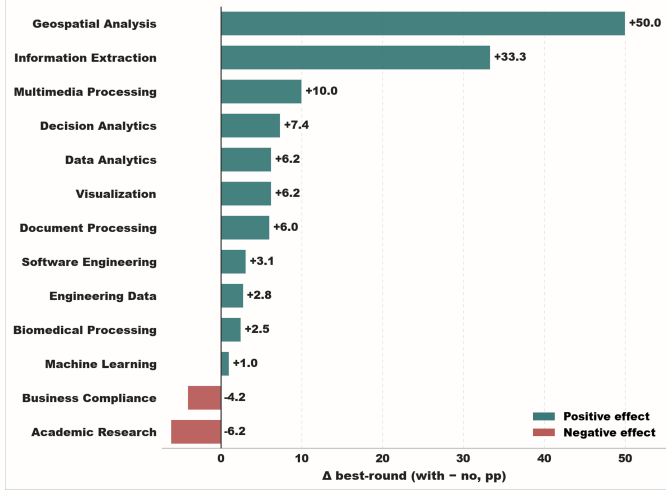
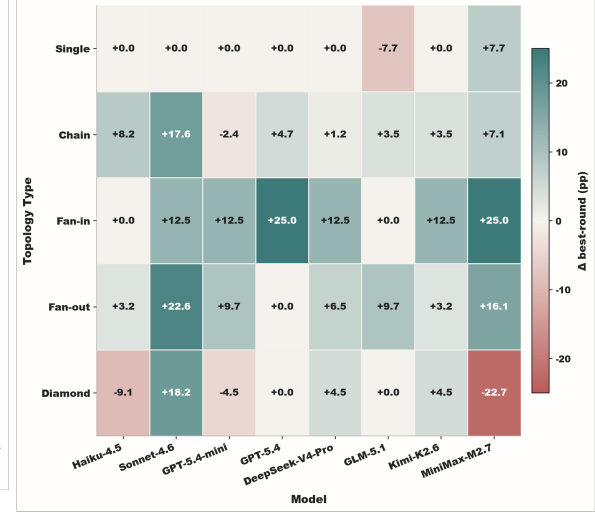


Figure 8 Failure Types



(a) Skill Effect by Domain



(b) Skill Effect by Topology Type

Figure 9 Skill utility across domains and task topologies. (a) **Domain-wise Skill Effect:** The bar chart reports the average performance delta of skill augmentation over the no skill setting. (b) **Topology-wise skill effect:** The heatmap shows the mean delta for each topology across eight models, revealing that skill effectiveness depends jointly on task structure and model capability.

highlight the importance of hybrid retrieval and orchestration strategies for robust skill composition in the wild.

Analysis of Failure Types. To analyze bad cases, defined as tasks for which injecting a skill strictly reduces agent accuracy, an initial round of manual inspection is conducted and then extended with AI-assisted analysis. The resulting failures are organized into two level categories (Figure 8(c)). (1) Knowledge Bias (59.6%). This category covers cases in which the injected skill provides incorrect or misleading content. Representative patterns include tool misguidance, such as recommending an inappropriate library, version, or API; parameter misspecification, such as using an incorrect threshold, regular expression, or comparison direction; and input selection errors, where the wrong record or sub-table is selected under the skill’s decision rule. Related failures also arise from workspace-scope leakage, where hard-coded paths in the skill trigger reads or writes outside the target task workspace, and budget exhaustion, where extra subroutines introduced by the skill consume the step or token budget and ultimately truncate execution. These results suggest that **skill construction should be adaptive, so as to reduce the mismatch between human curated skills and the model’s actual execution behavior. However, effective self-adaptation typically requires a dynamic evaluation benchmark to guide the direction of optimization. In this regard, SkillNet-Gym can serve as a particularly useful testbed.** (2) Instruction Override (40.4%). In this failure mode, the injected skill overrides

requirements explicitly specified in the task prompt. This typically manifests as output-format drift, where the generated result follows an incorrect JSON schema, data type, or sign convention; preprocessing omission, where a required normalization or editing step is skipped; and template carryover, where scaffolding introduced by the skill is only partially completed. This pattern suggests that **current foundation models still struggle to balance contextual knowledge against direct instruction following**. More capable next generation models may need to internalize skill utilization as a more native capability, rather than treating skills as external additions that can conflict with task instructions.

Skill Benefits Are Highly Domain Dependent. Skill utility varies considerably across domains (Figure 9(a)). Tasks with highly procedural workflows, such as Geospatial Analysis (+50.0 pp) and Information Extraction (+33.3 pp), benefit the most, suggesting that reusable execution patterns can be effectively encapsulated into skills. Moderate gains are observed in analytics and visualization tasks, where skills mainly reduce tool use overhead while leaving substantial reasoning to the model. In contrast, Academic Research (-6.2 pp) and Business Compliance (-4.2 pp) exhibit slight performance degradation. These domains typically require open-ended exploration, adaptive planning, and dynamic evidence integration, where rigid procedural skills may prematurely constrain the agent’s search strategy. Overall, the results indicate that the effectiveness of skills is strongly influenced by the procedural regularity of the underlying domain rather than being uniformly beneficial across all tasks.

Task Topology Determines Skill Effectiveness. Task topology exhibits a clear correlation with skill effectiveness (Figure 9(b)). Single skill tasks receive almost no benefit, indicating that isolated procedures can already be handled reliably by modern foundation models. As dependency structures become more compositional, performance improvements increase substantially. Chain, Fan-out, and especially Fan-in topologies consistently obtain the largest gains across most models, suggesting that skills are particularly effective at encoding reusable coordination patterns across multiple dependent subtasks. Diamond tasks, however, display the greatest variance, with strong models benefiting considerably while weaker models occasionally suffering performance degradation. This observation indicates that executing highly compositional skills requires not only well-structured procedural knowledge but also sufficient instruction-following and coordination capabilities from the underlying model. Overall, skills can provide their greatest value when tasks involve complex compositions of reusable operations rather than isolated actions.

Case Study: From SkillNet to Synthesized Tasks We provide an example of the conversion from SkillNet to task instructions, as shown in Figure 10.

Furthermore, SkillNet-Gym can continuously synthesize new benchmarks and evolve together with the skill ecosystem.

6 SkillNet-Fabric

6.1 Overview

SkillNet-Gym shows that agents can benefit from curated skills, but identifying a compact skill set whose capabilities jointly support a task remains difficult in large community skill pools. We therefore introduce **SkillNet-Fabric**, a task time routing layer that places a task specific Wiki between the global skill ecosystem and final set formation. The Wiki brings together task relevant candidates and their supporting evidence, which the Explorer uses to form the final skill set. Figure 11 summarizes this process from the global Wiki to the task specific routing context.

6.2 Routing Task and Routing Space

Let $\mathcal{S} = \{s_i\}_{i=1}^N$ denote the global SkillNet ecosystem. Given a task q and selection budget K , a router π returns a final set $\mathcal{Y}_q$. The objective is joint task support, meaning that the capabilities and instructions in $\mathcal{Y}_q$ should collectively cover the requirements expressed by q .

Before selecting $\mathcal{Y}_q$, the router determines a candidate set $\mathcal{S}_q$ and evidence $\mathcal{E}_q$ available for evaluating it.

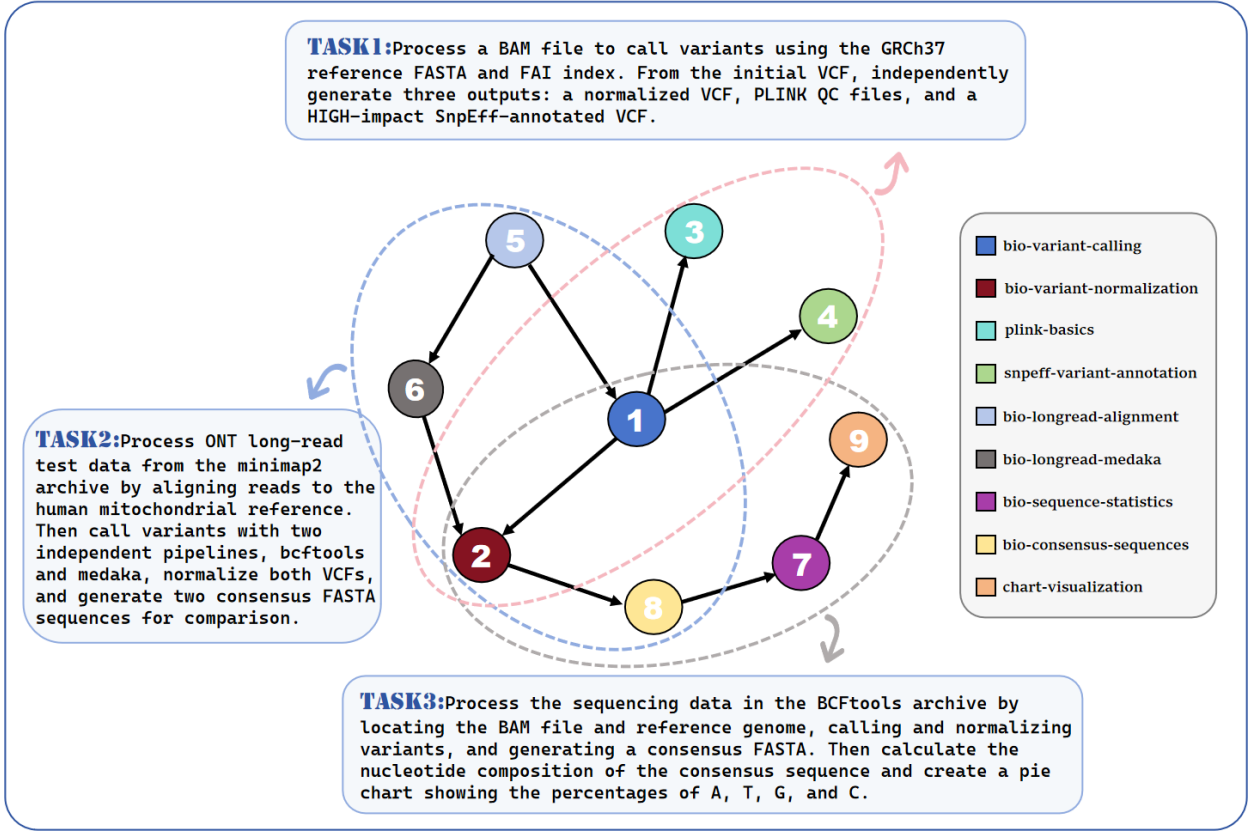


Figure 10 Case Study: From SkillNet subgraphs to synthesized tasks.

Together, these components define the routing space $\mathcal{R}_q$. The final set is then formed from this space:

$$\begin{aligned} \mathcal{R}_q &= (S_q, \mathcal{E}_q), & S_q &\subseteq S, \\ \mathcal{Y}_q &= \pi(q, \mathcal{R}_q; K), & \mathcal{Y}_q &\subseteq S_q, \quad |\mathcal{Y}_q| \leq K. \end{aligned} \quad (1)$$

Recovering the required set therefore has two aspects: candidate availability, which determines the contents of S_q , and final set formation, through which the Explorer constructs $\mathcal{Y}_q$ from those candidates. Expanding S_q can make more required skills available, but it can also introduce irrelevant candidates and context, making final set formation less focused [57].

Routing methods construct the routing space in different ways. Figure 12 compares two routing paradigms with SkillNet-Fabric. Direct retrieval ranks and reranks candidates by task relevance, while graph retrieval also uses relations to expand or organize them [42, 58]. SkillNet-Fabric combines retrieval and relation expansion to construct S_q , then materializes the retained candidates and evidence as a task specific Wiki. The Explorer navigates this Wiki to form the final set $\mathcal{Y}_q$.

6.3 Build Time Skill Representation

During the build stage, heterogeneous skill sources are converted into reusable, source grounded representations. This stage produces contracts C , a typed relation graph $\mathcal{G}$, retrieval indexes $\mathcal{J}$, and a global Wiki $\mathcal{W}$:

$$(C, \mathcal{G}, \mathcal{J}, \mathcal{W}) = \text{Build}(S). \quad (2)$$

Skill Contracts. For each skill, its stated capability, use conditions, inputs, and outputs are extracted into a source grounded contract. The contract provides a compact representation for retrieval and relation construction, while its fields remain linked to supporting source lines.

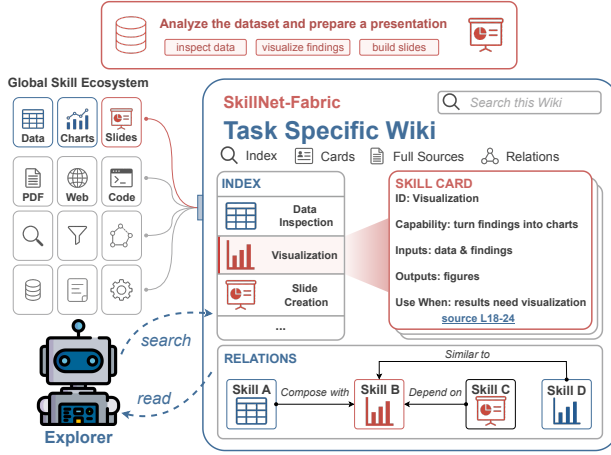


Figure 11 Framework of SkillNet-Fabric. A task specific Wiki provides the Explorer with retained candidates and evidence for final skill selection.

Methods	Recall $\uparrow$	FullCoverage $\uparrow$
$K = 5$		
BM25	30.00	20.00
BGE Large v1.5	54.40	37.33
Qwen3 E×R	61.40	42.67
SkillRouter	63.40	46.67
SkillNet-Fabric	67.91	49.33
$K = 10$		
BM25	36.50	24.00
BGE Large v1.5	61.10	41.33
Qwen3 E×R	67.77	49.33
SkillRouter	69.42	50.67
SkillNet-Fabric	74.80	60.00

Table 5 Skill routing on SkillRouter Hard. Recall and FullCoverage are macro percentages over 75 tasks. Qwen3 E×R denotes Qwen3 Embedding × Qwen3 Ranker.

Typed Relations. Relation construction first identifies likely related skill pairs from explicit references, compatible inputs and outputs, and lexical or semantic proximity. A language model then reads the two skill sources and classifies each pair as `depend_on`, `compose_with`, or `similar_to`. Dependency and composition relations preserve their source and target roles, whereas similarity is symmetric and identifies possible alternatives. The resulting typed graph organizes the Wiki and supports candidate discovery and comparison.

Global Wiki. The global Wiki organizes the skill ecosystem into a navigable knowledge space grounded in the underlying skill sources. Its linked pages represent each skill with a card that summarizes its contract and links to the complete source, while an index connects these pages for navigation. Together, these pages provide the structured context that routing uses to project the task specific Wiki for the Explorer.

6.4 Task Time Wiki Localization

At task time, routing uses the retrieval indexes and typed relation graph to identify the candidate set and evidence available for a task. Relevant seeds are retrieved first and then expanded through graph relations. The retained candidates and evidence are projected from the global Wiki to form a task specific Wiki $\mathcal{W}_q$:

$$\begin{aligned} (\mathcal{S}_q, \mathcal{E}_q) &= \text{Localize}(q; \mathcal{G}, \mathcal{J}, L), \\ \mathcal{W}_q &= \text{Project}(\mathcal{W}; \mathcal{S}_q, \mathcal{E}_q). \end{aligned} \quad (3)$$

The parameter L sets the maximum number of candidates admitted to $\mathcal{W}_q$.

Hybrid Seed Retrieval. BM25 captures exact procedural terms [59], whereas the dense index captures semantic matches. Reciprocal rank fusion combines the two rankings [60], and the fused ranking determines the seed set for relation expansion.

Relation Expansion. Starting from the seed set, relation expansion follows typed relations up to a fixed graph depth and admits connected candidates until L candidates have been retained. This step supplements task relevance with related skills that may not appear among the initial retrieval results. Seeds are retained first, and expanded candidates fill the remaining capacity.

Wiki Projection. The task specific Wiki projects the retained candidates from the global Wiki and augments them with the retrieval and relation evidence used to admit them. Local relations support comparison among the retained skills.

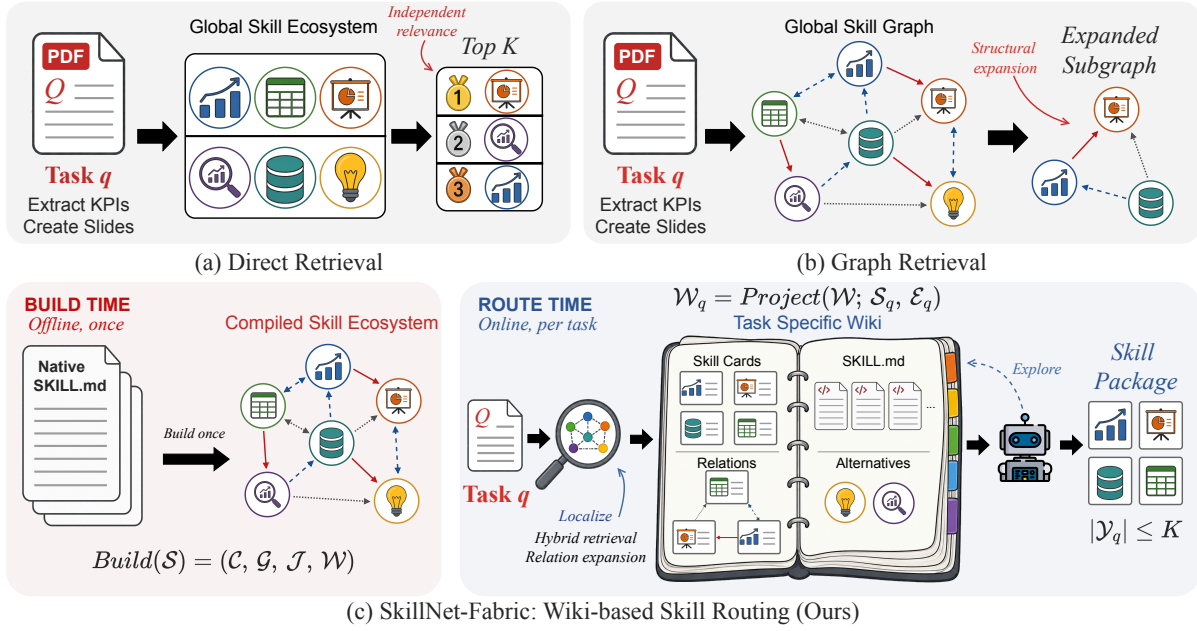


Figure 12 Comparison of two routing paradigms with SkillNet-Fabric.

6.5 Wiki Exploration and Route Formation

Wiki Exploration. The Explorer turns the task specific Wiki into a final skill set. It starts from the Wiki index and contract cards, selectively reads complete sources, and consults local relations when candidates require comparison. Based on this evidence, it selects up to K skills and records how each selected skill contributes to the task. The selected set is

$$\mathcal{Y}_q = \text{Explore}(q, \mathcal{W}_q; K). \quad (4)$$

Route Formation. The Planner combines the original task with the selected skills and their supporting evidence to produce an execution prompt for the downstream agent.

6.6 Experimental Settings

Benchmarks and Metrics. We evaluate SkillNet-Fabric at two levels: routing quality on SkillRouter Hard [42] and downstream task performance on SkillsBench [38] and AgentSkillOS [44]. SkillRouter Hard contains 75 scored tasks over 79,141 candidate skills. We report macro Recall of required skills and FullCoverage, the fraction of tasks for which the complete required set is selected. SkillsBench contains 87 tasks with deterministic verifiers; its metric is the task macro mean verifier reward. AgentSkillOS contains 30 tasks across five categories and reports task level Bradley–Terry scores [61] for skill pools of 53, 500, and 1,000 skills.

Baselines. For SkillRouter Hard, the comparison includes BM25 [59], BGE Large v1.5 [62], Qwen3 Embedding with Qwen3 Ranker [63], and the official SkillRouter method [42]. SkillsBench compares SkillNet-Fabric with execution without skills, Golden Skills, SkillRouter, and SkillsVote [64]; Golden Skills denotes the reference skill set. AgentSkillOS compares SkillNet-Fabric with execution without skills, exposure to the full skill pool, and the native AgentSkillOS system.

Configuration. For task time routing, we use BGE M3 embeddings [65] and expand relations to depth two. On SkillRouter Hard, GPT-5.6 Terra [66] serves as the Explorer with 100 retrieval seeds, $L = 1,000$, and $K \in \{5, 10\}$. SkillsBench uses the same localization configuration with $K = 10$. For downstream evaluation, we use GPT-5.4 mini [67] and GPT-5.6 Terra, with the same model performing routing, planning, and execution within each condition. AgentSkillOS uses eight seeds, $L = 100$, and $K = 8$.

(a) SkillsBench: Mean Verifier Reward (%)				
Methods	Easy ↑	Medium ↑	Hard ↑	Overall ↑ [95% CI]
GPT-5.4 mini				
w/o Skills	47.22	27.73	29.46	29.63 [22.54, 37.09]
Golden Skills	48.14	50.13	32.20	44.22 [35.89, 52.51]
SkillRouter	42.59	42.18	40.54	41.68 [33.26, 50.21]
SkillsVote	40.74	34.91	33.93	34.99 [27.30, 42.93]
SkillNet-Fabric	62.96	48.45	37.98	46.08 [37.51, 54.62]
GPT-5.6 Terra				
w/o Skills	52.77	46.06	45.24	46.26 [37.12, 55.53]
Golden Skills	48.14	66.17	52.92	60.66 [51.90, 69.29]
SkillRouter	53.70	59.66	46.49	55.01 [45.82, 63.95]
SkillsVote	59.26	60.34	50.42	57.07 [47.89, 65.89]
SkillNet-Fabric	69.44	67.92	63.69	66.67 [58.75, 74.58]
(b) AgentSkillOS: Bradley–Terry Score				
Methods	Seed Pool ↑	500 Skills ↑	1,000 Skills ↑	Pool Mean ↑
GPT-5.4 mini				
w/o Skills	33.16	42.16	46.96	40.76
Full Pool	53.85	50.99	46.30	50.38
AgentSkillOS	61.11	46.96	53.13	53.73
SkillNet-Fabric	61.89	65.20	65.54	64.21
GPT-5.6 Terra				
w/o Skills	34.42	30.03	39.34	34.60
Full Pool	28.38	46.29	34.28	36.32
AgentSkillOS	40.84	44.95	40.43	42.07
SkillNet-Fabric	90.03	78.53	80.45	83.00

Table 6 Downstream task performance on SkillsBench and AgentSkillOS. Panel (a) reports task macro mean verifier rewards over 87 SkillsBench tasks; brackets contain 95% confidence intervals for the Overall score. Panel (b) reports task level Bradley–Terry scores rescaled to 0 to 100 and averaged over 30 AgentSkillOS tasks across pools of 53, 500, and 1,000 skills.

6.7 Main Results

Routing Quality on SkillRouter Hard. SkillNet-Fabric achieves the highest required skill recall and complete set recovery among the compared methods at both selection budgets. Table 5 reports macro Recall and FullCoverage when the router selects from 79,141 candidate skills. At $K = 5$, SkillNet-Fabric reaches 67.91% Recall and 49.33% FullCoverage, exceeding the official SkillRouter method by 4.51 and 2.66 percentage points, respectively. Increasing the budget to $K = 10$ raises the scores to 74.80% and 60.00%, widening the margins to 5.38 and 9.33 points. The wider FullCoverage margin suggests that additional selection capacity is particularly useful for recovering complementary skills needed for complete set formation.

Downstream Task Performance. We next evaluate whether the selected routes improve downstream performance. Table 6 reports results on SkillsBench and AgentSkillOS. On SkillsBench, SkillNet-Fabric achieves the highest overall verifier reward in both model conditions: 46.08% with GPT-5.4 mini and 66.67% with GPT-5.6 Terra. It exceeds SkillRouter by 4.40 percentage points in the former condition and SkillsVote by 9.60 points in the latter, the strongest routing baselines in their respective settings. On AgentSkillOS, SkillNet-Fabric ranks first in all six combinations of model and skill pool, with Pool Mean scores of 64.21 for GPT-5.4 mini and 83.00 for GPT-5.6 Terra. Under GPT-5.4 mini, its margin over the strongest comparator grows from 0.78 points on the seed pool to 14.21 and 12.41 points on the pools of 500 and 1,000 skills.

Variant	Wiki Components				Downstream Quality		Exploration
	Graph Expansion	Relation Evidence	Contract Cards	Complete Sources	BT Score	Δ	Tokens (K)
w/o Graph Expansion	✗	✓	✓	✓	54.15	−11.36	16.66
w/o Relation Evidence	✓	✗	✓	✓	53.05	−12.46	30.26
w/o Contract Cards	✓	✓	✗	✓	49.80	−15.71	34.23
w/o Complete Sources	✓	✓	✓	✗	48.12	−17.39	24.42
Full Wiki	✓	✓	✓	✓	65.51	0.00	27.82

Table 7 Wiki component ablation on AgentSkillOS. “✓” and “✗” indicate whether each component is present or removed. Δ is relative to the full Wiki. Explorer tokens are averaged per task and reported in thousands.

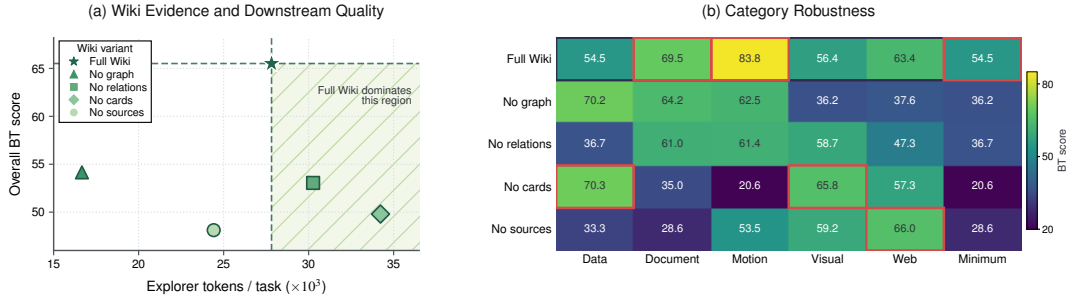


Figure 13 Effects of Wiki components on AgentSkillOS. Panel (a) relates overall Bradley–Terry score to Explorer tokens per task. Panel (b) reports category scores and the minimum across categories.

6.8 Analysis

Wiki Components. To isolate the contribution of individual Wiki components to downstream task performance, we compare the complete Wiki with variants that remove graph expansion, relation evidence, contract cards, or complete sources. The evaluation covers all 30 AgentSkillOS tasks with GPT-5.4 mini and the pool of 500 skills; Table 7 reports the overall Bradley–Terry score and mean Explorer tokens for each variant.

The complete Wiki obtains a Bradley–Terry score of 65.51, at least 11.36 points above every ablation. The ablations reveal that downstream quality does not follow Explorer token use alone. Removing contract cards or relation evidence increases token use to 34.2K and 30.3K per task, respectively, while scores fall to 49.80 and 53.05. Removing graph expansion or complete sources reduces token use to 16.7K and 24.4K, but scores also fall to 54.15 and 48.12. The relationship between quality and Explorer tokens in Figure 13(a) therefore shows that the Wiki’s benefit depends on the composition of the available context. Graph expansion and complete sources affect which candidates can be inspected and how they can be assessed, while cards and relation evidence provide compact cues for navigating and comparing them.

The category results in Figure 13(b) show that the components contribute differently across task categories. The variant without graph expansion remains strong on Data (70.2) but is weak on Visual (36.2) and Web (37.6). The variant without contract cards produces a similar contrast, with 70.3 on Data and 65.8 on Visual but only 20.6 on Motion. The variant without complete sources leads on Web (66.0) while falling to 33.3 on Data and 28.6 on Document. In contrast, the complete Wiki maintains a minimum category score of 54.45, compared with 36.73 for the strongest minimum among the ablations. No individual component helps every task type uniformly; the complete Wiki gives the most stable coverage across categories.

Candidate Availability and Final Set Formation. We further study how the localization limit affects candidate availability and final set formation on the 75 SkillRouter Hard tasks. We vary $L \in \{10, 100, 1,000, 5,000, 10,000\}$ and compare these settings with the full pool. For each task and value of L , GPT-5.6 Luna, GPT-5.6 Terra, and GPT-5.6 Sol receive the same task specific Wiki and select at most ten skills. Candidate FullCoverage measures whether every required skill is available before exploration, whereas Final FullCoverage@10 measures whether

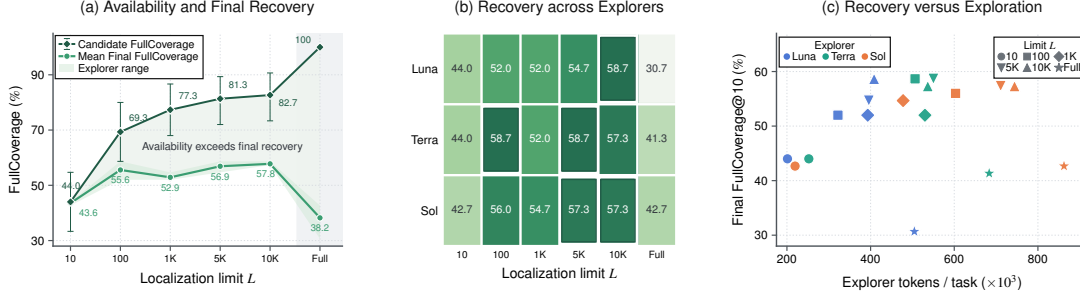


Figure 14 Candidate availability and final set formation on SkillRouter Hard. Panel (a) compares Candidate FullCoverage with mean Final FullCoverage@10 as L grows. Panel (b) reports Final FullCoverage@10 by Explorer, and panel (c) relates Final FullCoverage@10 to Explorer tokens. Full denotes all 79,141 skills.

the Explorer forms the complete required set. Figure 14 summarizes the resulting relationship between candidate availability, final set formation, and exploration effort. The sweep separates candidate availability from final set formation. Candidate FullCoverage rises from 44.00% at $L = 10$ to 69.33% at $L = 100$, 81.33% at $L = 5,000$, and 82.67% at $L = 10,000$, with only a 1.33 point increase in the last interval. The full pool makes all required skills available and reaches 100% Candidate FullCoverage. Final FullCoverage@10 follows a different trajectory: the mean increases from 43.56% at $L = 10$ to 57.78% at $L = 10,000$, then drops to 38.22% with the full pool. At $L = 10$, the two measures are nearly identical, so most failures reflect limited candidate availability. Under the full pool, their 61.78 point gap shows that the remaining failures arise during final set formation, when the Explorer compares candidates. Figure 14(a) visualizes this separation.

Results across Explorers show that the preferred value of L varies by model. Luna improves as the candidate space grows and reaches 58.67% at $L = 10,000$; Terra reaches 58.67% at both $L = 100$ and $L = 5,000$; and Sol peaks at 57.33% at both $L = 5,000$ and $L = 10,000$. For each Explorer, the best tested L setting outperforms the corresponding full pool condition. Figure 14(b) shows these differences across Explorers. With the routing procedure and selection budget held fixed, the different peaks suggest that the preferred scope depends on the Explorer model’s capacity to compare the available candidates.

Exploration effort follows the same pattern. At each Explorer’s best tested L setting, token use is lower and Final FullCoverage@10 is higher than under the full pool, as shown in Figure 14(c). A larger context can therefore increase Explorer token use without improving complete set recovery. Taken together, the results support the two stage routing design. Increasing L improves candidate availability, while an excessively broad space adds candidates and context that can make final set formation less focused. The task specific Wiki connects these stages by presenting the Explorer with the candidates and evidence retained during routing, and its most effective scope depends on the Explorer model in the tested settings.

7 Application Scenarios

As illustrated in Figure 15, SkillNet bridges the gap between high-level user intentions and executable agent actions by organizing specialized skills into a coherent workflow. We detail below its specific utility in autonomous scientific discovery and coding agent scenarios. Note that the examples are illustrative prototypes and do not represent real-world applications.

7.1 SkillNet for Autonomous Scientific Discovery

In this section, we demonstrate how SkillNet organizes heterogeneous agent skills into a coherent, executable, and evaluable research workflow. This system performs the scientific task of identifying potential disease-associated genes and candidate therapeutic targets from large-scale biological data, ultimately generating research reports characterized by scholarly readability.

SkillNet first schedules data processing skills to perform cleaning and clustering on single-cell RNA-seq data, thereby identifying key genetic markers. Subsequently, SkillNet invokes mechanistic analysis and target validation skills to map these genes onto biological pathways and cross-validate their clinical significance. Finally, SkillNet activates report generation skills to integrate fragmented analytical results into structured scientific documents with formal citations.

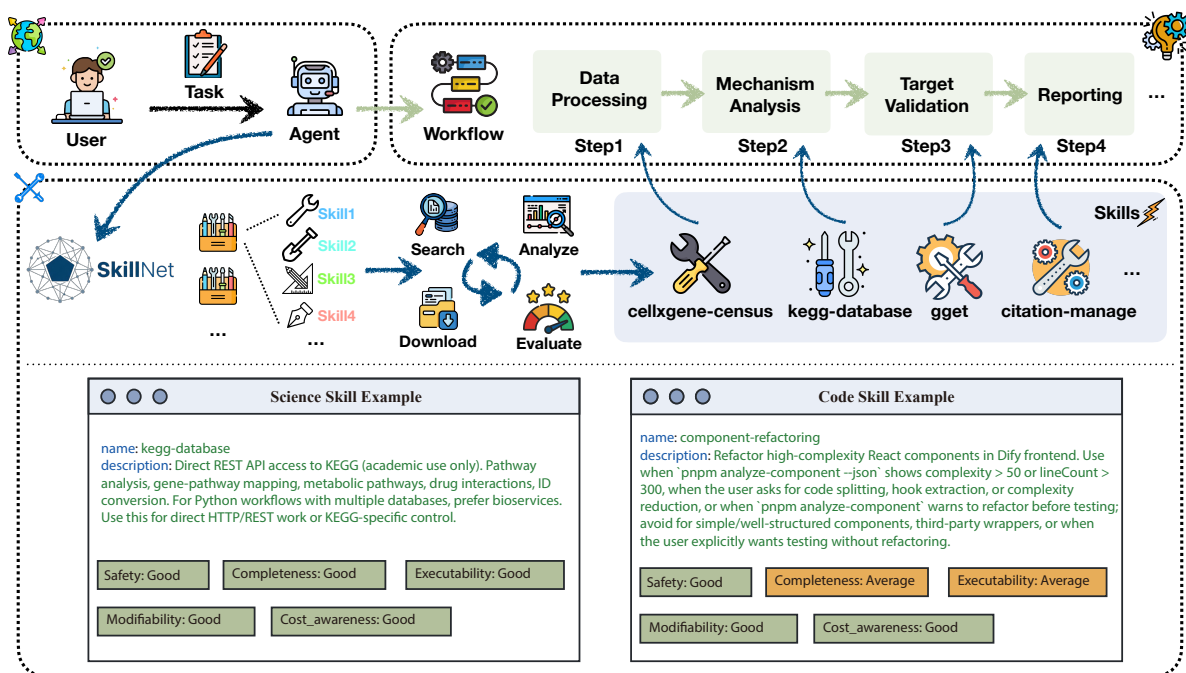


Figure 15 Example of SkillNet application scenarios. The framework decomposes user task into actionable steps (top), with representative skill acquisition and multi-dimensional evaluations for Science and Coding scenarios (bottom).

This process profoundly reflects the core value of SkillNet: transforming fragmented research skills into a structured, composable network of capabilities. This architecture enables an AI Scientist to transcend the limitations of single-domain tasks and engage meaningfully in the complete closed-loop process of complex scientific discovery.

7.2 SkillNet for Autonomous Coding Agent

In this section, we construct a coding scenario tailored for large-scale software engineering tasks to demonstrate the potential of SkillNet in complex code comprehension, refactoring, and functional development. The objective of this scenario is to execute feature expansion and structural evolution within an existing production-grade codebase while ensuring correctness, performance, and long-term maintainability.

The workflow begins with SkillNet scheduling code analysis skills to construct a structured representation of the system architecture. Subsequently, it coordinates requirements decomposition and impact analysis skills to precisely map high-level functional requirements to specific code-level modification targets while assessing regression risks. During the implementation phase, SkillNet organizes generation, testing, and validation skills into a closed-loop iterative process, utilizing real-time feedback to drive automated corrections. Finally, it invokes maintenance skills to generate architectural update documentation characterized by complete traceability.

This scenario illustrates the core value of SkillNet: remodeling complex software engineering challenges as problems of capability composition and coordination. By restructuring fragmented skills such as coding, analysis, and testing into a structured network of agent skills, SkillNet provides a reliable and scalable automation paradigm for complex engineering tasks, enabling AI to competently perform system-level software evolution.

7.3 Using OpenClaw with SkillNet

In this section, we demonstrate how SkillNet integrates with OpenClaw², a highly customizable open-source personal AI agent framework. OpenClaw adopts a lazy-loading skill design: at session initialization, only compact skill metadata is injected into the system prompt, and full instructions are loaded on demand

²<https://github.com/openclaw/openclaw>.

when triggered. SkillNet is integrated as a skill in this framework, extending the agent with dynamic skill acquisition, quality-aware library management, and experience-driven knowledge creation.

Once installed, SkillNet introduces three core behavioral patterns that are activated automatically based on conversational context. First, when a user poses a complex or unfamiliar task, the agent performs a pre-task search against the SkillNet repository to locate relevant skills. It then downloads the matched skills into the local workspace and reads the retrieved SKILL.md to extract applicable patterns and tool configurations. In addition, when the user provides a GitHub repository URL or shares a document such as a PDF, the agent can directly invoke skillnet create to generate a structured skill from that source, making the external knowledge immediately available for the current task. Second, when the user requests skill library organization or quality auditing, the agent invokes SkillNet’s analyze and evaluate commands to generate structured reports, which include multi-dimensional quality scores along with inter-skill relationship graphs that capture similar_to, belong_to, compose_with, and depend_on edges. Third, after completing a task whose solution involved non-obvious or reusable knowledge, or when the user explicitly requests experience consolidation, the agent proactively invokes skillnet create to package the solution as a standardized skill and runs automatic evaluation to verify its quality before admitting it into the repository.

This integration establishes a closed loop where community-contributed skills inform task execution, successful outcomes are consolidated into new skills, and periodic analysis maintains repository quality. Rather than treating each conversation as an isolated episode, the OpenClaw + SkillNet combination transforms a general-purpose agent into a continuously self-improving system through structured skill circulation, directly supporting the vision of scalable, evolving agent intelligence.

8 Related Work

8.1 Experience Consolidation and Skill Abstraction

LLM-based agents have been extended with tools, planning, and memory to perform long-horizon tasks in complex environments [3, 4, 5, 6, 27]. Several works have explored how agents can acquire and refine skills from interaction and experience. Reflexion [19] and Expel [20] investigate how agents can summarize failures and extract corrective feedback. Memory-centric methods aim to accumulate long-term experience to support continual learning [68, 69, 70, 71]. Other studies explore structured skill learning and cross-task reuse, promoting generalization and transfer across domains [24, 72, 73, 74, 75, 76, 77, 78]. Despite these advances, most approaches still represent skills implicitly by encoding them in prompts, latent memories, or loosely organized workflows, thereby hindering systematic consolidation, evaluation, and reuse.

8.2 Skill Repositories and Evaluations

Parallel to the development of agentic systems, a number of community-driven skill repositories [25, 26, 79, 80] and evaluations [38, 81, 82] have emerged to curate agent skills, aiming to promote reuse, standardization, and ecosystem-level collaboration. For instance, platforms like ClawHub function as npm-like version management hubs, while SkillsMP and Skills.sh act as expansive open-source directories aggregating GitHub repositories. Additionally, SkillHub introduces premium marketplace features with predefined skill stacks and basic rating systems. Recently, Li et al. [83] introduce SkillsBench, an 87-task, 11-domain benchmark showing that curated agent Skills significantly boost LLM agent performance (+16.2 pp on average) while self-generated Skills offer no gain, revealing that models benefit from consuming but cannot reliably author procedural knowledge. These platforms provide valuable resources for sharing prompts, tool wrappers, and task-solving templates.

As summarized in Table 8, existing platforms predominantly operate as static package managers or marketplaces, facing three critical limitations: First, they largely rely on manual curation and ad hoc quality control, lacking automated mechanisms to generate skills dynamically from agent trajectories or existing codebases. Second, their evaluation practices primarily focus on simple community metrics (e.g., repository stars) or end-task performance, failing to offer comprehensive insights into intrinsic properties such as safety, completeness, executability, maintainability, and cost-awareness. Third, these skill collections often suffer from redundancy, brittleness, and poor composability because skills are treated as isolated entities, restricting their scalability in large agent populations.

These limitations motivate the need for a unified framework for skill organization, verification, and evolution. In contrast to distribution-centric platforms, our proposed **SkillNet** provides a full-lifecycle

Dimension	SkillNet (Ours)	ClawHub	SkillsMP	SkillHub	Skills.sh
Core Positioning	Full-lifecycle infrastructure (creation, evaluation, & connection)	npm-like version management hub	Massive open-source ecosystem catalog	Premium marketplace (curated & stacked)	Universal skill directory & leaderboard
Automated Creation	Supported (generates skills from trajectories, code, and documents via LLMs)	Not supported	Not supported	Not supported	Not supported
Quality Evaluation	Multi-dimensional metrics (safety, completeness, executability, maintainability, cost-aware)	None (relies on highlighted recommendations)	Basic filtering (e.g., GitHub stars)	Built-in LLM rating (e.g., S-rank/B-rank)	Community leaderboard (downloads/trends)
Relationship Analysis	Supported (extracts structural relations to build a skill graph)	Not supported	Not supported	Manual pre-configured skill stacks	Not supported
Integration & Distribution	Python SDK & CLI tool (skillnet-ai)	npx-based package manager (clawhub install)	GitHub repository aggregation & CLI	Desktop client 1-click install & CLI	Universal CLI (npx skills add)
Search Mechanism	Vector semantic search + keyword matching	Vector semantic search + version control	AI semantic search + keyword matching	Semantic search + ranking filters	Global directory search
Skill Quantity	600k+ Total, 500k+ Curated	~9k+	~261k+	~21k+	~71k+

Table 8 Comparison of SkillNet and Existing AI Agent Skill Platforms

infrastructure. It addresses the aforementioned bottlenecks through automated skill creation via LLM pipelines, rigorous multi-dimensional evaluation, and relational connectivity that weaves isolated tools into a structured Skill Graph, thereby offering comprehensive foundational support for self-evolving agent ecosystems.

9 Conclusion, Discussion and Future Work

In this work, we introduced SkillNet, an open infrastructure for creating, evaluating, and organizing AI skills at scale. By systematically consolidating experience, structuring skills, and providing principled evaluation, SkillNet enables agents to improve cumulatively, perform reliably across tasks, and adapt to complex, open-ended environments. This framework lays the foundation for scalable continual learning and robust skill composition, bridging the gap between raw model capability and sustained, evolving intelligence.

Conceptually, SkillNet is grounded in a unified view of three complementary constraints on an agent’s generative capability: workflows, memory, and skills. Workflows impose explicit procedural structure, ensuring reliability but remaining inherently rigid. Memory accumulates contextual experience and associative knowledge, enabling adaptation but lacking operational boundaries. Skills bridge these extremes by packaging reusable capability units that both constrain generation and organize memory into actionable patterns. In this perspective, skills serve as the structured interface through which memory becomes executable and workflows become flexible.

Looking ahead, the emerging paradigm of the “one-person company” or “one-person lab” envisions a single expert orchestrating a society of agents. Through SkillNet, skills become the primary unit of knowledge integration and delegation: individuals curate skill repositories, agents compose them into workflows, and memory continuously refines them through experience. This closed loop transforms isolated automation into cumulative machine expertise, enabling minimal human teams to achieve organizational-scale intelligence.

Open-World Skill Evolution. Achieving automatic skill discovery, abstraction, and cross-domain transfer in open-world settings remains highly challenging. In domains such as **industrial manufacturing**, **finance**,

and scientific research, this requires the dynamic composition and optimization of complex tasks. An industry-specific, privately curated SkillNet may itself become a foundational component of the agent infrastructure. Moreover, integrating skill evolution mechanisms with online feedback, causal reasoning, and uncertainty modeling holds promise for improving the reliability of skill selection.

Model-Skill Synergy. Although SkillNet provides agents with large-scale executable skills, the synergy between these skills and the underlying model capabilities remains largely unexplored. In particular, the question of how to leverage neuro-symbolic integration and memory mechanisms to enable skill structures to guide model decision paths and to dynamically restructure skill hierarchies and dependencies as model capabilities evolve remains central to systematic study.

Multi-Agent Collaboration and Knowledge Sharing. In multi-agent environments, SkillNet can function as a shared representation and exchange layer, supporting collaborative planning, knowledge transfer, and the accumulation of experience across agents. By continuously consolidating agent behaviors into reusable skills, SkillNet may further supports the emergence of **digital avatar** whose capabilities are progressively distilled from accumulated skills. More broadly, this paradigm may open a pathway toward collective intelligence, where skills evolve into transferable, composable units of coordination and where digital personas can inherit, share, and extend competencies beyond individual agents.

10 Limitations

There remain several limitations in the present work. First, the coverage of skills is inevitably incomplete. Many capabilities in private or specialized domains cannot be incorporated, while low-frequency or highly tacit abilities are difficult to capture and consolidate within the repository, particularly when they resist explicit linguistic description. Second, the quality of self-constructed skills cannot be fully guaranteed. Although our evaluation procedures filter out some problematic cases, a substantial portion of skills still lacks rigorous and systematic assessment. If malicious users contribute “poisoned” or adversarial skills, SkillNet’s current Safety evaluation mechanism can detect some of these cases, but cannot fully mitigate them. Third, an end-to-end pipeline that translates natural-language requirements into fully instantiated agents via SkillNet has not yet been established; this remains an important direction for future work. We will continue to curate, maintain, and improve the system over time.

11 Acknowledgement

We sincerely thank the open-source developer community on GitHub (e.g., Skill_Seekers³, Vercel_Skills⁴) for their broad and generous contributions to skills. We are grateful to Lu Chen, Xiang Li, Zhuosheng Zhang for their valuable suggestions and insightful feedback on this work.

References

- [1] David Silver and Richard S Sutton. Welcome to the era of experience. *Google AI*, 1, 2025.
- [2] Xiang Chen, Ningyu Zhang, Xin Xie, Shumin Deng, Yunzhi Yao, Chuanqi Tan, Fei Huang, Luo Si, and Huajun Chen. Knowprompt: Knowledge-aware prompt-tuning with synergistic optimization for relation extraction. In *Proceedings of the ACM Web conference 2022*, pages 2778–2788, 2022. URL <https://dl.acm.org/doi/10.1145/3485447.3511998>.
- [3] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. The rise and potential of large language model based agents: a survey. *Sci. China Inf. Sci.*, 68(2), 2025. URL <https://doi.org/10.1007/s11432-024-4222-0>.

³https://github.com/yusufkaraaslan/Skill_Seekers

⁴<https://github.com/vercel-labs/skills>

- [4] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In Sean Follmer, Jeff Han, Jürgen Steimle, and Nathalie Henry Riche, editors, *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST 2023, San Francisco, CA, USA, 29 October 2023- 1 November 2023, pages 2:1–2:22. ACM, 2023. URL <https://doi.org/10.1145/3586183.3606763>.
- [5] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2024. URL <https://openreview.net/forum?id=ehfRiFOR3a>.
- [6] Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory, 2023. URL <https://arxiv.org/abs/2305.17144>.
- [7] Runnan Fang, Xiaobin Wang, Yuan Liang, Shuofei Qiao, Jialong Wu, Zekun Xi, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Synworld: Virtual scenario synthesis for agentic action knowledge refinement. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 437–448. Association for Computational Linguistics, 2025. URL <https://doi.org/10.18653/v1/2025.acl-short.33>.
- [8] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023. URL <https://arxiv.org/abs/2303.18223>.
- [9] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A survey on in-context learning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 1107–1128. Association for Computational Linguistics, 2024. URL <https://doi.org/10.18653/v1/2024.emnlp-main.64>.
- [10] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *CoRR*, abs/2402.07927, 2024. URL <https://doi.org/10.48550/arXiv.2402.07927>.
- [11] Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, et al. Cl-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*, 2026.
- [12] Douglas L Hintzman. " schema abstraction" in a multiple-trace memory model. *Psychological review*, 93(4):411, 1986.
- [13] Rudi Studer, V. Richard Benjamins, and Dieter Fensel. Knowledge engineering: Principles and methods. *Data Knowl. Eng.*, 25(1-2):161–197, 1998. URL [https://doi.org/10.1016/S0169-023X\(97\)00056-6](https://doi.org/10.1016/S0169-023X(97)00056-6).
- [14] Clarence Irving Lewis, Cooper Harold Langford, and P Lamprecht. *Symbolic logic*, volume 170. Dover publications New York, 1959.
- [15] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)*, 51(3):1–39, 2018. URL <https://dl.acm.org/doi/10.1145/3182657>.
- [16] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. *ACM Computing Surveys (Csur)*, 54(4):1–37, 2021. URL <https://dl.acm.org/doi/10.1145/3447772>.
- [17] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015. URL <https://www.nature.com/articles/nature14539>.
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 770–778. IEEE Computer Society, 2016. URL <https://doi.org/10.1109/CVPR.2016.90>.

- [19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- [20] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pages 19632–19642. AAAI Press, 2024. URL <https://doi.org/10.1609/aaai.v38i17.29936>.
- [21] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025. URL <https://arxiv.org/abs/2504.07079>.
- [22] Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail. *arXiv preprint arXiv:2601.04748*, 2026. URL <https://arxiv.org/abs/2601.04748>.
- [23] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution, 2026. URL <https://arxiv.org/abs/2601.21557>.
- [24] Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. Cua-skill: Develop skills for computer using agent, 2026. URL <https://arxiv.org/abs/2601.21123>.
- [25] SkillsMP Team. Skillsmp: Claude skills marketplace. <https://skillsmp.com/>, 2026. Accessed: 2026-02-06.
- [26] SkillHub Team. Skillhub: A marketplace for claude and agent skills. <https://www.skillhub.club/>, 2026. Accessed: 2026-02-06.
- [27] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- [28] Anthropic. Agent skills: An open standard for modular ai agent capabilities. <https://agentskills.io/>, 2025. Accessed: 2026-02-05.
- [29] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning, 2021. URL <https://arxiv.org/abs/2010.03768>.
- [30] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023. URL <https://arxiv.org/abs/2207.01206>.
- [31] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent smarter than a 5th grader?, 2022. URL <https://arxiv.org/abs/2203.07540>.
- [32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [33] Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization for llm agents, 2024. URL <https://arxiv.org/abs/2403.02502>.
- [34] DeepSeek-AI. Deepseek-v3 technical report. <https://github.com/deepseek-ai/DeepSeek-V3>, 2024. Accessed: 2026-02-07.
- [35] Google DeepMind. Gemini 2.5 technical report and model card (referencing gemini 2.5 pro). URL <https://deepmind.google/models/gemini/pro/>. Accessed: 2026-02-07.
- [36] OpenAI. Openai o4-mini model documentation. <https://platform.openai.com/docs/models>, 2025. Accessed: 2026-02-07.

- [37] Anthropic. Announcing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, 2026. Accessed: 2026-04-26.
- [38] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026. URL <https://arxiv.org/abs/2602.12670>.
- [39] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering?, 2026. URL <https://arxiv.org/abs/2603.15401>.
- [40] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks, 2026. URL <https://arxiv.org/abs/2604.20087>.
- [41] Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. How well do agentic skills work in the wild: Benchmarking llm skill usage in realistic settings, 2026. URL <https://arxiv.org/abs/2604.04323>.
- [42] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Skill routing for llm agents at scale, 2026. URL <https://arxiv.org/abs/2603.22455>.
- [43] Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. Skillret: A large-scale benchmark for skill retrieval in llm agents, 2026. URL <https://arxiv.org/abs/2605.05726>.
- [44] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026. URL <https://arxiv.org/abs/2603.02176>.
- [45] Yifan Zhou, Zhentao Zhang, Ziming Cheng, Shuo Zhang, Qizhen Lan, Zhangquan Chen, Zhi Yang, Qianyu Xu, Ronghao Chen, Huacan Wang, and Sen Hu. Skillgenbench: Benchmarking skill generation pipelines for llm agents, 2026. URL <https://arxiv.org/abs/2605.18693>.
- [46] Jiahao Ying, Boxian Ai, Wei Tang, Siyuan Liu, and Yixin Cao. Openskilleva: Automatically auditing the open skill ecosystem for llm agents, 2026. URL <https://arxiv.org/abs/2605.23657>.
- [47] Anthropic. Claude Code: an agentic coding tool. <https://github.com/anthropics/claude-code>, 2025.
- [48] OpenAI. Codex CLI: Lightweight coding agent that runs in your terminal. <https://github.com/openai/codex>, 2025.
- [49] MiniMax. The minimax-m2 series: Mini activations unleashing max real-world intelligence. *CoRR*, abs/2605.26494, 2026. doi: 10.48550/ARXIV.2605.26494. URL <https://doi.org/10.48550/arXiv.2605.26494>.
- [50] Kimi Team. Kimi K2.5: visual agentic intelligence, 2026. URL <https://doi.org/10.48550/arXiv.2602.02276>.
- [51] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [52] GLM. GLM-5: from vibe coding to agentic engineering. *CoRR*, abs/2602.15763, 2026. doi: 10.48550/ARXIV.2602.15763. URL <https://doi.org/10.48550/arXiv.2602.15763>.
- [53] Yusuf Karaaslan. Skill seekers, 2026. URL https://github.com/yusufkaraaslan/Skill_Seekers. Repository for converting documentation websites, GitHub repositories, and PDFs into Claude-compatible skills.
- [54] Le Chen, Erhu Feng, Yubin Xia, and Haibo Chen. Skvm: Revisiting language vm for skills across heterogenous llms and harnesses, 2026. URL <https://arxiv.org/abs/2604.03088>.
- [55] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. Coevoskills: Self-evolving agent skills via co-evolutionary verification, 2026. URL <https://arxiv.org/abs/2604.01687>.
- [56] Shuzheng Si, Haozhe Zhao, Yu Lei, Qingyi Wang, Dingwei Chen, Zhitong Wang, Zhenhailong Wang, Kangyang Luo, Zheng Wang, Gang Chen, Fanchao Qi, Minjia Zhang, and Maosong Sun. From context to skills: Can language models learn from context skillfully?, 2026. URL <https://arxiv.org/abs/2604.27660>.

- [57] Zifei Wang, Wei Wen, Qiang Ji, Keyu Chen, Ruizhi Qiao, and Xing Sun. Skill is not document: Query-conditioned compatibility for llm agent skill routing, 2026. URL <https://arxiv.org/abs/2606.03565>.
- [58] Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. Graph-of-skills: Dependency-aware structural retrieval for massive agent skills. In First Workshop on Agent Skills, 2026. URL <https://openreview.net/forum?id=HfGDY8mV67>.
- [59] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. Found. Trends Inf. Retr., 3(4):333–389, April 2009. ISSN 1554-0669. doi: 10.1561/15000000019. URL <https://doi.org/10.1561/15000000019>.
- [60] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '09, page 758–759, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584836. doi: 10.1145/1571941.1572114. URL <https://doi.org/10.1145/1571941.1572114>.
- [61] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. Biometrika, 39(3/4):324–345, 1952. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2334029>.
- [62] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '24, page 641–649, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704314. doi: 10.1145/3626772.3657878. URL <https://doi.org/10.1145/3626772.3657878>.
- [63] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176>.
- [64] Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. Skillsvote: Lifecycle governance of agent skills from collection, recommendation to evolution, 2026.
- [65] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Findings of the Association for Computational Linguistics: ACL 2024, pages 2318–2335, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.137. URL <https://aclanthology.org/2024.findings-acl.137/>.
- [66] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, July 2026. Accessed: 2026-07-29.
- [67] OpenAI. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>, March 2026. Accessed: 2026-07-29.
- [68] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025, volume 267 of Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025. URL <https://proceedings.mlr.press/v267/wang25bx.html>.
- [69] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL <https://arxiv.org/abs/2508.06433>.
- [70] Guibin Zhang, Muxin Fu, and Shuicheng Yan. Memgen: Weaving generative latent memory for self-evolving agents, 2025. URL <https://arxiv.org/abs/2509.24704>.
- [71] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>.
- [72] Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library, 2025. URL <https://arxiv.org/abs/2512.17102>.

- [73] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.
- [74] Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. Polyskill: Learning generalizable skills through polymorphic abstraction, 2025. URL <https://arxiv.org/abs/2510.15863>.
- [75] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent kb: Leveraging cross-domain experience for agentic problem solving, 2025. URL <https://arxiv.org/abs/2507.06229>.
- [76] Yue Wu, Tianhao Su, Shunbo Hu, and Deng Pan. Skill-based autonomous agents for material creep database construction, 2026. URL <https://arxiv.org/abs/2602.03069>.
- [77] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.08234>.
- [78] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents, 2026. URL <https://arxiv.org/abs/2602.02474>.
- [79] Vercel. Skills.sh: The open agent skills ecosystem. <https://skills.sh/>, 2026. Accessed: 2026-02-06.
- [80] OpenClaw. Clawhub: The skill dock for sharp agents. <https://clawhub.ai/>, 2026. Accessed: 2026-02-22.
- [81] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. Agent skills in the wild: An empirical study of security vulnerabilities at scale, 2026. URL <https://arxiv.org/abs/2601.10338>.
- [82] George Ling, Shanshan Zhong, and Richard Huang. Agent skills: A data-driven analysis of claude skills for extending large language model functionality, 2026. URL <https://arxiv.org/abs/2602.08004>.
- [83] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. arXiv preprint arXiv:2602.12670, 2026. URL <https://arxiv.org/abs/2602.12670>.

Appendix

A Comparison Traits Details

We compare representative agentic skill benchmarks along eight traits spanning three dimensions: benchmark construction, benchmark scope, and evaluation focus.

- **Real-World Source:** Whether the benchmark is grounded in artifacts, repositories, or tasks drawn from real-world or community environments, rather than being primarily hand-crafted, synthetic, or narrowly curated for evaluation convenience.
- **Automatic Construction:** Whether the benchmark can be constructed largely through an automated pipeline, such as automatic data collection, graph construction, task synthesis, or verifier generation, instead of relying mainly on manual authoring and curation.
- **Dynamic Extensibility:** Whether the benchmark can be incrementally updated as new skills, artifacts, or workflows emerge, allowing it to evolve with the underlying ecosystem rather than remaining a static snapshot.
- **Domain Coverage:** Whether the benchmark covers a sufficiently broad range of domains and task settings, enabling evaluation beyond a narrow vertical such as software engineering or a small fixed taxonomy.
- **Skill Pool:** Whether the benchmark includes a comparatively large candidate pool of skills or skill-like artifacts, such that agents must operate under realistic retrieval or selection conditions rather than relying only on pre-attached or task-specific skills.
- **Task Numbers:** The number of benchmark task instances available for evaluation. Larger task sets typically improve statistical reliability, diversity, and robustness of conclusions, although scale alone does not imply realism or breadth.
- **Skill Construction:** Whether the benchmark explicitly evaluates an agent’s ability to create, generate, or refine new skills, rather than only consuming pre-existing skills during task solving.
- **Skill Composition:** Whether the benchmark explicitly evaluates how agents retrieve, organize, or compose multiple skills or tool-like units into coherent workflows, rather than treating skills as isolated single-use aids.
- **Skill Execution:** Whether the benchmark evaluates end-to-end task solving with executable environments and deterministic verification, focusing on whether skill use ultimately improves real execution outcomes.

B Further Experiment Details

B.1 Construction Baselines

We additionally evaluate SkillSeeker [53] under skill construction setting. The results are reported in Table 9. The results suggest that different skill construction methods can play complementary roles across models. Meanwhile, both methods largely rely on directly compressing documentation into reusable skills, their effectiveness is inherently limited. This observation points to a promising direction for future work: skill construction methods that go beyond static distillation and instead support iterative self-improvement and empirical validation.

B.2 End-to-End Performance in the Wild

We evaluate agents in a wild setting, where they are exposed to a large skill library containing over 2000 skills, and measure their task performance. The results are shown in Table 10. Compared with the setting where the agent is provided with the correct official skills, performance drops on both Easy and Hard tasks to varying degrees, with the degradation being particularly pronounced on Hard tasks. We attribute this

decline to the difficulty of operating over a massive skill library: the agent often fails to retrieve relevant skills, or retrieves only an incomplete subset of them. These results further highlight that a strong skill composition baseline is crucial for enabling effective skill usage in realistic, open-world scenarios.

Models	Skill Creator		Skill Seeker	
	Easy	Hard	Easy	Hard
Claude Code				
Minimax-M2.7	66.88	20.73	60.39	23.17
Kimi-K2.6	73.38	31.10	75.32	38.41
Deepseek-V4	68.83	29.88	72.73	23.17
GLM-5.1	79.22	31.10	70.78	28.05

Table 9 Performance of skill construction baselines.

Models	Official Skill		In Wild	
	Easy	Hard	Easy	Hard
Claude Code				
Minimax-M2.7	65.58	26.22	65.58	23.17
Kimi-K2.6	74.03	34.15	66.23	25.61
Deepseek-V4	69.44	29.87	68.83	24.39
GLM-5.1	71.43	32.93	66.23	24.39

Table 10 End-to-End Task Performance in the Wild Setting.

C Benchmark Details

C.1 Benchmark Construction Details

SkillNet Construction Details. We implement SkillNet construction as a staged pipeline that turns broad community skill retrieval results into a scenario-mediated directed skill graph. The pipeline first collects and materializes candidate skills, removes near-duplicates, abstracts each skill into pre- and post-scenarios, aligns compatible scenarios across skills, and finally aggregates verified alignments into `compose_with` edges for task synthesis.

Skill Retrieval and Materialization. SkillNet construction starts from broad query seeds covering diverse domains. For each query, we call the SkillNet search API in vector mode with a fixed similarity threshold and bounded top- k limit. Retrieved skills are ranked by GitHub stars and truncated to the top candidates, with optional deduplication by skill URL or name. The selected `skill_urls` are then resolved and downloaded from GitHub, producing a local skill corpus with a manifest that records provenance and download status.

Near-Duplicate Skill Removal. To reduce redundancy, each retained skill is embedded using its name, metadata, matched queries, and full `SKILL.md` content. A FAISS nearest-neighbor index identifies highly similar skills, which are merged with union-find clustering. The representative of each cluster is selected by GitHub stars with deterministic tie-breaking.

Scenario Extraction and Canonicalization. For each deduplicated skill, an LLM extracts scenario-mediated transitions from the skill specification. A pre-scenario describes the state before the skill is applicable, while a post-scenario describes the state after successful execution. Raw scenarios are normalized, embedded, and organized into a sparse FAISS similarity graph. Louvain community detection provides coarse groups, which are further refined by complete-linkage clustering. Each final cluster is assigned a canonical scenario name and linked back to its skills.

Scenario Alignment and Edge Verification. Cross-skill edge candidates are induced by retrieving compatible pre-scenarios for each post-scenario. Post-scenario embeddings query a FAISS index over pre-scenario embeddings, and candidates below the retrieval threshold are discarded. An LLM judge then verifies whether the source post-scenario can naturally satisfy the target pre-scenario in a real workflow.

Graph Assembly. The graph builder aggregates verified post-to-pre alignments into directed `compose_with` edges between skill nodes. Multiple alignments between the same source-target skill pair are grouped into one edge with accumulated evidence. Each edge stores the source and target skills, supporting scenarios, alignment types, confidence scores, retrieval scores, and scenario-level evidence. The final directed SkillNet graph is used for subgraph sampling and downstream task synthesis.

Quality Control for SkillNet. To ensure that SkillNet remains reliable and non-redundant, we apply quality control throughout its construction, covering skills, edges and the downstream artifacts. 1) *Skill Level:* Each downloaded `SKILL.md` is evaluated by an LLM judge along three dimensions: environment cost, output verifiability, and documentation quality. A skill is retained only if it meets the minimum threshold for each dimension and also satisfies the overall weighted score requirement. We further remove near-duplicate skills by embedding the full skill descriptions, retrieving high-similarity neighbors with FAISS, clustering

duplicates, and keeping the highest-star representative within each cluster. 2) *Edge Level*: Candidate skill pairs are reviewed using both their full SKILL.md files and scenario-level evidence. We remove edges when the target skill merely restates the source capability, represents an alternative implementation of the same function, or fails to introduce a substantively new state transition. 3) *Artifact Level*: Auxiliary documents and files are retained only if they are substantively relevant to the associated skill, which helps filter out loosely related or noisy context. Together, these steps keep SkillNet compact, well-grounded, and effective for downstream task synthesis.

Quality Control for Synthesized Tasks. Task-level quality control covers four core components 349 of each synthesized task: the instruction, the refer- 350 ence solution, the test script, and the task difficulty.

Human-Aligned Rubric Judge for Task Instructions. We design a rubric-based judge to assess the quality of synthesized task instructions along three key dimensions: 1) *Verifiability*, whether the instruction admits a uniquely testable outcome under the evaluation script; 2) *Human-expert Realism*, whether the instruction resembles one that a human expert would naturally write, rather than exhibiting an overly synthetic or machine-generated style; and 3) *DAG compliance*, whether the instruction is consistent with the workflow specified by the sampled DAG. To further align the judge with authentic human preferences, we refine the prompting strategy of the LLM-as-a-Judge using manually curated benchmarks, such as SkillsBench, as supervision signals.

PRM-Based Verification of Ground-Truth Solutions. Although we obtain candidate solutions by prompting an agent with meta guidance, the resulting outputs are often difficult to verify directly and may otherwise require costly review by human experts. To address this issue, we employ an additional agent to monitor and assess the intermediate execution trajectory used to produce the solution. If this verifier agent judges the reasoning and execution process to be sound, we treat the resulting artifact as a trustworthy ground-truth solution. Importantly, we do not immediately discard samples whose trajectories are flagged as problematic, as doing so would incur substantial synthesis cost. Instead, we feed the failure signals and diagnostic feedback from unsuccessful attempts into the context of subsequent retries. This iterative refinement process continues until a valid solution is produced or a maximum retry budget is reached, at which point the sample is abandoned.

Test Script Generation with Post-hoc Verification. The synthesized evaluation script should satisfy two key requirements: it must be fully passed by the reference solution, and it must comprehensively cover all evaluation conditions implied by the task instruction. We iteratively refine the test script until these criteria are satisfied. In addition, to prevent the tests from being either too weak or overly restrictive, we design a rubric that allows an LLM to evaluate the quality of the test script itself. We also conduct preliminary rollouts with candidate models to assess whether the tests are under-constraining and fail to distinguish correct solutions from incomplete or incorrect ones.

Difficulty-Aware Filtering and Human Cross-Validation. To ensure that the synthesized tasks genuinely challenge an agent’s ability to orchestrate complex skill workflows, we adopt a rigorous two-stage task retention pipeline. First, we apply difficulty-aware filtering to remove tasks that can be successfully solved by GLM-5.1 [52] in three independent trials without skills. This step removes shortcut-prone tasks that can be completed from general prior knowledge alone. The remaining tasks are then independently reviewed by three annotators with at least master’s-level training in computer science. Each annotator evaluates instruction clarity, ground-truth correctness, and test-script reliability. Disagreements are resolved by majority vote after discussion. Inter-annotator agreement is high, with Fleiss’s Kappa $\kappa = 0.834$, indicating strong consistency within the range $0.80 \leq \kappa \leq 1.00$.